龙信磐石平台对接文档

版本号：V1.0.0
更新日期：2026年06月25日

---

1. 概述

1.1 平台简介

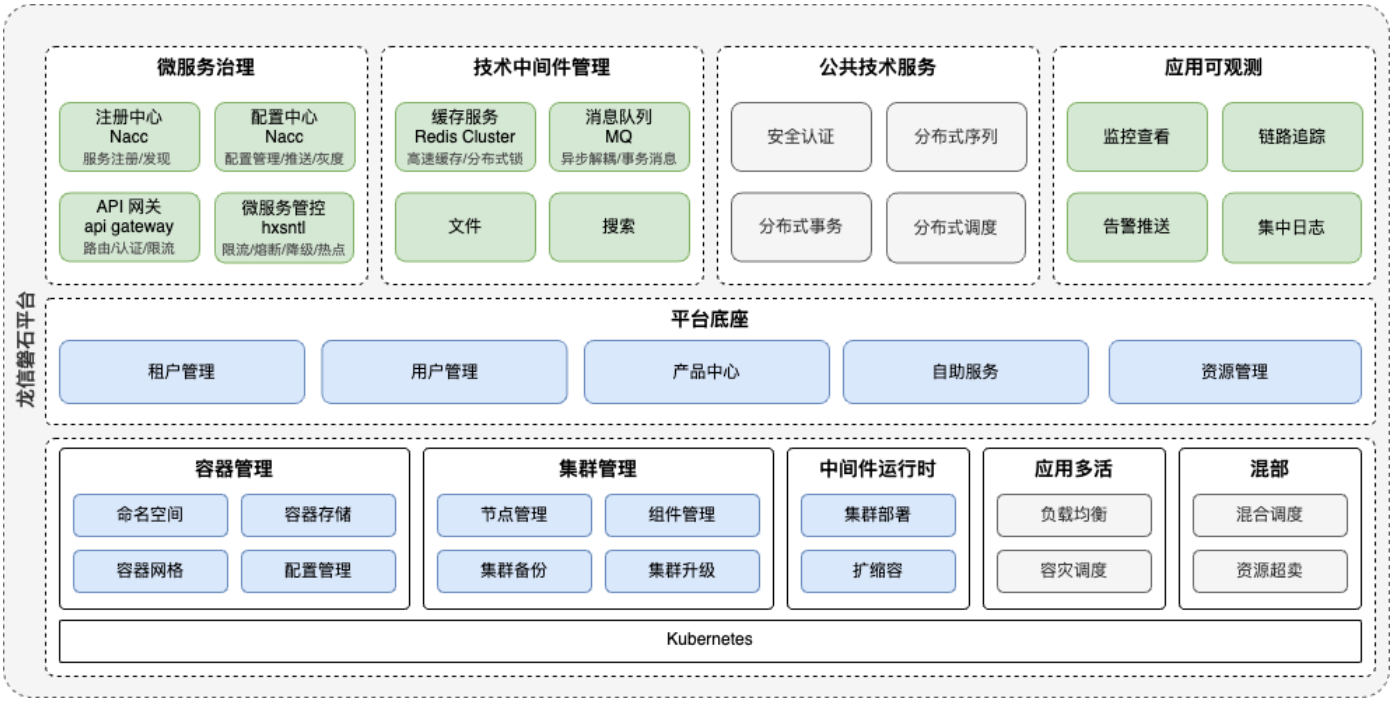
1.1.1 龙信磐石平台定位与能力

龙信磐石平台（简称磐石平台）是企业级 PaaS（Platform as a Service）平台，旨在为业务系统提供一站式的分布式中间件服务能力，降低微服务架构下的开发运维门槛。平台以容器化为基础，封装了微服务管理、API网关管理、中间件、可观测性等关键能力，业务系统只需关注自身业务逻辑，无需自行搭建和运维底层基础设施。

磐石平台核心定位：

- **中间件服务化**：将注册中心、配置中心、缓存、消息队列等中间件以服务形式对外提供，业务系统通过标准 SDK 接入
- **微服务治理**：提供流量控制、熔断降级、热点参数限流、系统保护
- **API网关管理**：提供API列表管理、访问控制、流量控制、网关路由
- **可观测体系**：整合日志、指标、链路追踪三位一体的监控能力
- **运维自动化**：支持服务启停、日志采集、故障自愈等运维操作

1.1.2 核心功能架构图



1.1.3 适用场景

- **微服务架构转型**：传统单体应用拆分为微服务，需要统一的服务治理能力
- **多环境隔离需求**：测试生产、生产等多套环境需要隔离管理
- **分布式系统建设**：需要注册中心、配置中心、消息队列等分布式组件支持
- **可观测性建设**：需要统一的日志、监控、链路追踪能力

1.2 名词术语

名词	说明
系统（Tenant）	租户在磐石平台的资源隔离和权限管理的顶层隔离单位
项目（Project）	系统下的项目分组，用于区分同一系统的不同版本或模块（如"订单系统-V1"、"订单系统-V2"）
工作空间（Workspace）	项目下的环境隔离单元，dev、sit、uat、prod
数据中心（Datacenter）	磐石平台中的地理隔离维度，用于区分服务部署的物理位置
API网关（API Gateway）	统一流量入口，负责请求路由、认证授权、流量控制等
命名空间（Namespace）	配置中心的隔离维度，用于区分不同环境或不同业务线的配置
分组（Group）	配置的数据组织单元，按业务功能或模块对配置进行分组管理
DataId	配置的唯一标识，用于精确定位某条配置
服务名（ServiceName）	注册中心中服务的标识，服务提供者和服务消费者通过服务名进行发现

---

2. 前期准备

2.1 业务系统平台使用申请

业务系统接入磐石平台前，需完成系统开通及产品资源申请。磐石平台采用“ITIL申请 + 平台资源开通”的管理模式，确保平台资源统一管理和规范使用。

业务系统接入流程如下：

```
系统申请
    ↓
系统开通
    ↓
产品申请
    ↓
产品资源分配
    ↓
系统接入实施
```

2.1.1 系统申请

系统是租户在磐石平台进行资源隔离和权限管理的基本单位。应用系统首次接入磐石平台时，需先完成系统申请。

申请方式如下：

1. 登录 ITIL 服务管理平台；
2. 提交“【1149】磐石平台_系统开通及变更”流程；
3. 按要求填写系统申请信息；
4. 提交流程审批。

审批通过后，由磐石平台管理员完成系统创建，并分配对应管理权限。

2.1.2 产品申请

系统开通完成后，业务系统可根据实际需求申请使用磐石平台相关产品能力（注册中心、配置中心、缓存、消息中间件、API网关、微服务管控、可观测平台

等），产品申请分为 ITIL申请和平台申请两个阶段。

（1）ITIL资源申请

登录 ITIL 服务管理平台，提交“【1150】磐石平台_产品申请及变更”流程。

（2）磐石平台产品申请

ITIL流程审批通过后，由申请人在磐石平台测试生产环境提交产品开通申请。

测试生产环境地址（开发测试网打开）：http://pdcp.hxbtest:8080

操作路径：服务申请与变更->产品申请

2.1.3 产品资源开通

磐石平台管理员根据审批结果完成产品资源创建及分配，并完成工单审批。

资源开通完成后，申请人可登录磐石平台查看相关产品资源信息。

产品信息查询路径：产品中心->产品配置中心 。

2.2 环境要求

2.2.1 网络要求

业务系统服务器需要能够访问磐石平台各中间件服务的网络地址，具体要求如下：

中间件	必需性	需要的网络策略
注册中心	必须	允许访问注册中心端口（默认18662）
配置中心	必须	允许访问配置中心端口（默认18662）
API 网关	必须	允许访问网关地址（按实际分配地址）
Redis	必须	允许访问 Redis 服务端口
CloudMQ-H	必须	允许访问 MQ 服务端口
可观测组件	必须	允许访问可观测服务端口（kafka/kibana端口）

> **网络开通方式**：联系网络管理相关人员，通过工单系统申请开通网络策略。

2.2.2 开发环境准备

项目	要求	说明
JDK	JDK 8 或 JDK 17+	推荐使用 JDK 17，性能更优
构建工具	Maven 3.6.3+	用于引入 SDK 依赖
IDE	IntelliJ IDEA / Eclipse	推荐 IntelliJ IDEA

---

3. 产品使用

3.1 注册中心

3.1.1 功能说明

注册中心基于 Nacc 实现，提供服务注册与发现能力，是微服务架构的基础组件。

服务发现机制

- **客户端主动注册**：服务提供者启动时通过 SDK 向注册中心注册自身服务实例
- **客户端主动注销**：服务停止时主动从注册中心注销，避免无效请求
- **主动拉取服务列表**：服务消费者启动时从注册中心拉取可用服务列表，并按一定周期刷新
- **订阅服务变更**：服务消费者订阅服务列表变更，注册中心推送变更通知

健康检查

- **心跳机制**：服务实例定期向注册中心发送心跳，续约服务实例 TTL（默认启用）
- **异常剔除**：注册中心检测到心跳超时（默认 5 秒）的实例，将其从可用列表中剔除
- **客户端上报**：应用可通过 SDK 主动上报自身健康状态

负载均衡策略

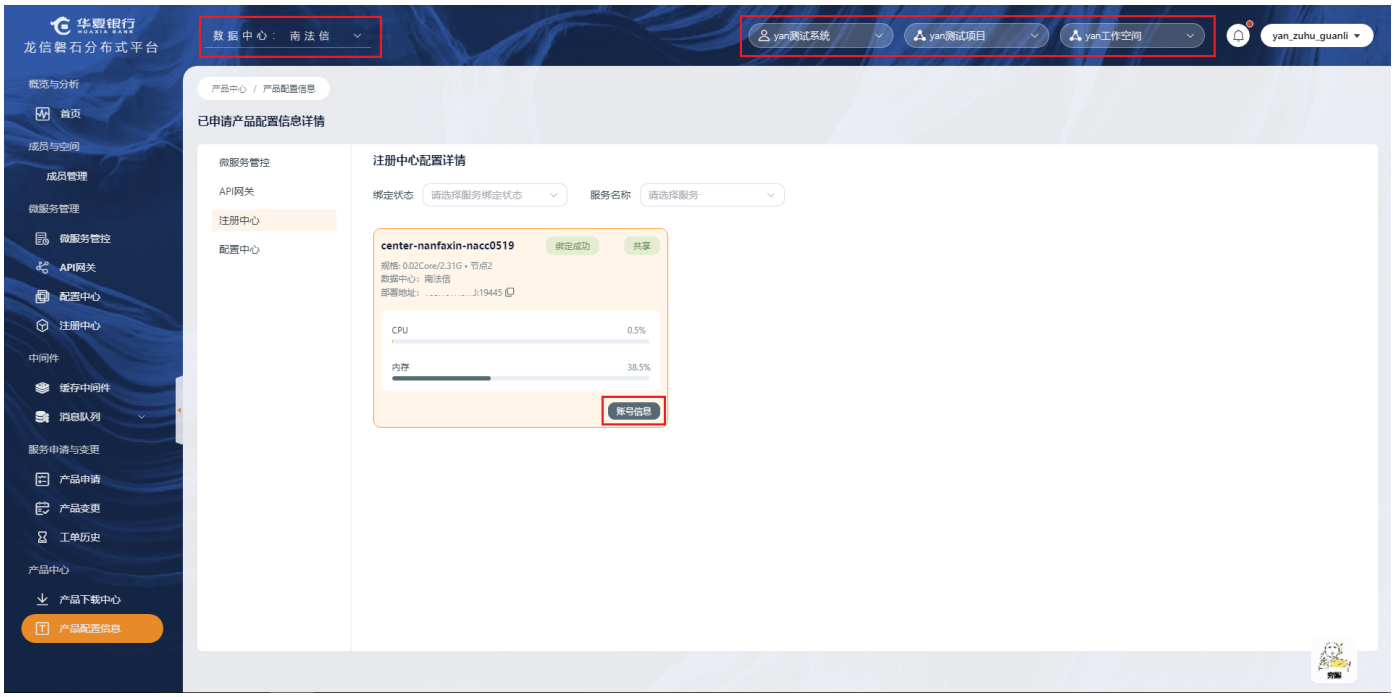
注册中心本身不提供负载均衡，负载均衡由服务消费者侧完成或通过 API 网关实现。常用策略：

- 随机策略
- 轮询策略
- 加权策略
- 最少活跃调用策略

3.1.2 接入配置

- 前置条件：完成注册中心产品申请
- 连接信息获取

系统管理员进入“产品配置信息-注册中心”详情页面，在已申请产品配置信息详情处选择注册中心，点击“账号信息”按钮。



分别复制账号、密码、命名空间、部署地址至注册中心客户端配置中的对应位置。



- 组件依赖

  应用项目pom.xml文件增加如下maven依赖：

```
<!-- 磐石平台注册中心 -->
<dependency>
  <groupId>com.hxb.cadp</groupId>
  <artifactId>spring-cloud-starter-hxnacc-discovery</artifactId>
  <version>***</version>
</dependency>
```

版本号兼容性说明：磐石平台注册中心客户端（spring-cloud-starter-hxnacc-discovery）或spring-cloud-alibaba提供的原生注册中心客户端（spring-cloud-starter-nacos-discovery）与springboot和jdk版本之间存在明确的版本兼容性。

1、使用磐石平台注册中心客户端

springboot版本	jdk版本	引入组件	使用版本
3.x	jdk17	spring-cloud-starter-hxnacc-discovery	2.1-RELEASE及以上版本
2.x	jdk1.8	spring-cloud-starter-hxnacc-discovery	1.0-RELEASE/1.1-RELEASE

2、使用原生spring-cloud-alibaba注册中心客户端

springboot版本	jdk版本	引入组件	使用版本
3.x	jdk17	spring-cloud-starter-nacos-discovery	2025.0.0.0
2.x	jdk1.8	spring-cloud-starter-nacos-discovery	2021.1

- 注册中心客户端配置

1、spring-cloud-starter-hxnacc-discovery 2.1-RELEASE 或 spring-cloud-starter-nacos-discovery 2025.0.0.0 application.yml 配置示例

```yaml
server:
  port: 8080
spring:
  application:
    name: test-server
  cloud:
    nacos:
      discovery:
        namespace: TENANT-PROJECT-WORKSPACE  # 命名空间ID，用于隔离不同的环境或项目，对应3.1.4章节中的“命名空间”
        server-addr: *.*.*.*:18662  # 注册中心服务地址，多个地址用逗号分隔，对应3.1.4章节中的“部署地址”
        username: nacos  # 访问认证用户名，对应3.1.4章节中的“账号”
        password: nacos  # 访问认证密码，对应3.1.4章节中的“密码”
        group: DEFAULT_GROUP  # 服务分组名称
```

> 注：
>
> 配置文件名要求为application.yml。

2、spring-cloud-starter-hxnacc-discovery 1.0-RELEASE/1.1-RELEASE 或spring-cloud-starter-nacos-discovery 2021.1 bootstrap.yml 配置示例

```yaml
server:
  port: 8080
spring:
  application:
    name: test-server
  cloud:
    nacos:
      username: nacos  # 访问认证用户名，对应3.1.4章节中的“账号”
      password: nacos  # 访问认证密码，对应3.1.4章节中的“密码”
      discovery:
        namespace: TENANT-PROJECT-WORKSPACE  # 命名空间ID，用于隔离不同的环境或项目，对应3.1.4章节中的“命名空间”
        server-addr: *.*.*.*:18662  # 注册中心服务地址，多个地址用逗号分隔，对应3.1.4章节中的“部署地址”
        group: DEFAULT_GROUP  # 服务分组名称
```

3.2 配置中心

3.2.1 功能说明

配置中心基于 Nacc 实现，提供配置管理和动态推送能力。

配置管理模型

概念	说明	示例
Namespace（命名空间）	配置的顶层隔离环境，对应不同环境或业务线	自动生成，通过磐石平台-“产品中心”-“产品配置信息”-“配置中心”-“已申请产品配置信息详情”-“配置中心”获取
Group（分组）	配置的数据组织单元，按业务功能组织	DEFAULT_GROUP
DataId	配置的唯一标识	`order-service.yaml` / `user-service.properties`

DataId 命名规范建议：`${service-name}-${env}.${file-type}`，例如 `order-service-prod.yaml`

配置推送机制

- **长轮询推送**：Nacc 采用长轮询机制，客户端定时请求服务端检查配置变更（默认 3 秒）
- **配置变更推送**：服务端检测到配置变更后，立即响应，客户端拉取最新配置
- **MD5 校验**：客户端通过 MD5 判断配置是否变更，减少无效传输

配置版本管理

- 每次配置变更自动生成新版本
- 支持查看历史版本并回滚
- 支持对比不同版本的差异

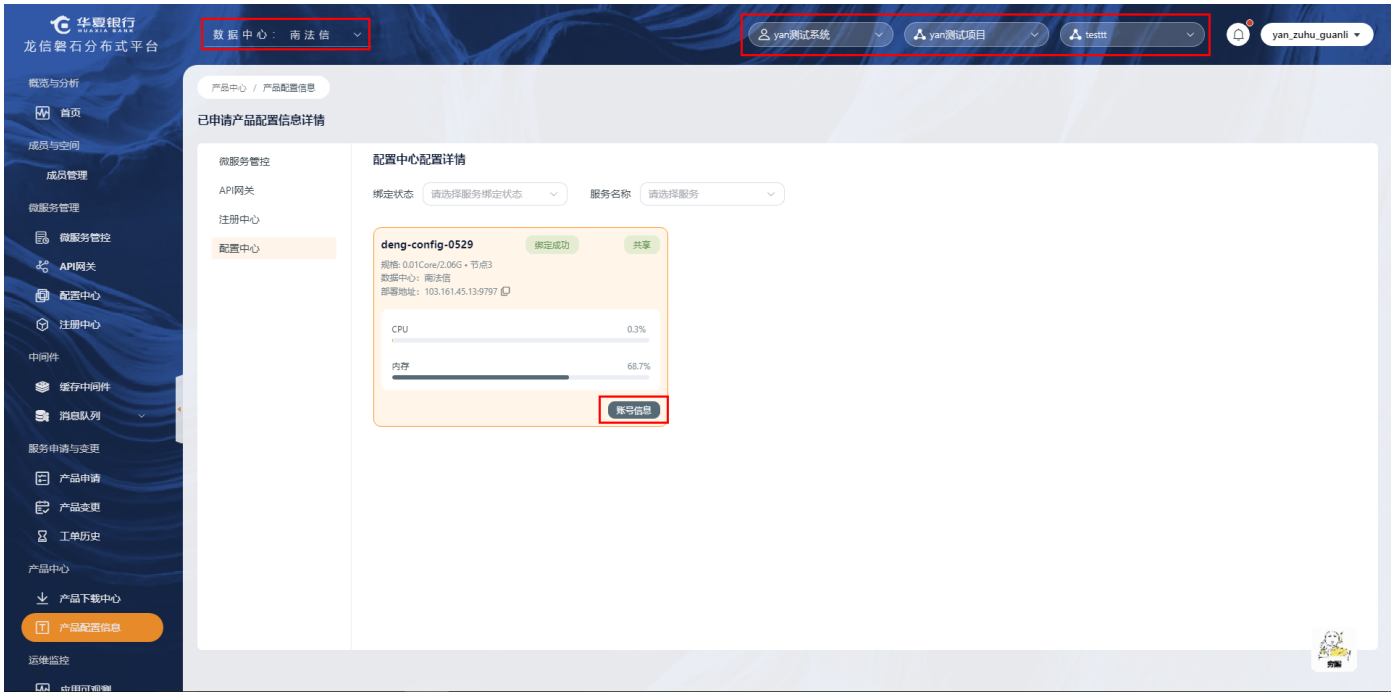
灰度发布

- 支持配置灰度规则，针对特定实例或用户推送不同配置
- 灰度规则支持百分比、IP白名单等条件

3.2.2 接入配置

- 前置条件：完成配置中心产品申请
- 连接信息获取

系统管理员进入“产品配置信息-配置中心”详情页面，在已申请产品配置信息详情处选择配置中心，点击“账号信息”按钮。



分别复制账号、密码、命名空间、部署地址至配置中心客户端配置中的对应位置。



- 组件依赖

  应用项目pom.xml文件增加如下maven依赖：

```
<!-- 磐石平台配置中心 -->
<dependency>
  <groupId>com.hxb.cadp</groupId>
  <artifactId>spring-cloud-starter-hxnacc-config</artifactId>
  <version>*</version>
</dependency>
```

> 注： 兼容性说明

> 磐石平台配置中心客户端（spring-cloud-starter-hxnacc-config）或spring-cloud-alibaba提供的原生配置中心客户端（spring-cloud-starter-nacos-config）与springboot和jdk版本之间存在明确的版本兼容性。
>
> 1、使用磐石平台配置中心客户端

springboot版本	jdk版本	引入组件	使用版本
3.x	jdk17	spring-cloud-starter-hxnacc-config	2.1-RELEASE及以上版本
2.x	jdk1.8	spring-cloud-starter-hxnacc-config	1.0-RELEASE/1.1-RELEASE

> 2、使用原生spring-cloud-alibaba配置中心客户端

springboot版本	jdk版本	引入组件	使用版本
3.x	jdk17	spring-cloud-starter-nacos-config	2025.0.0.0
2.x	jdk1.8	spring-cloud-starter-nacos-config	2021.1

- 配置中心客户端配置

1、spring-cloud-starter-hxnacc-config 2.1-RELEASE 或 spring-cloud-starter-nacos-config 2025.0.0.0 application.yml 配置示例

```
server:
  port: 8080
spring:
  application:
    name: test-server
  cloud:
    nacos:
      config:
        namespace: TENANT-PROJECT_WORKSPACE  # 命名空间ID，用于隔离不同的环境或项目，对应3.2.5章节中的“命名空间”
        server-addr: *.*.*.*:*  # 注册中心服务地址，多个地址用逗号分隔，对应3.2.5章节中的“部署地址”
        username: nacos  # 访问认证用户名，对应3.2.5章节中的“账号”
        password: nacos  # 访问认证密码，对应3.2.5章节中的“密码”
        group: DEFAULT_GROUP  # 服务分组名称
        file-extension: yml  # 配置文件类型
  # 新版推荐：显式声明从Nacos导入哪些Data ID
  # 版本推荐统一使用spring.config.import
  config:
    import:
      - optional:nacos:test1.yml?group=DEFAULT_GROUP&refreshEnabled=true
      - optional:nacos:test2.yml?group=DEFAULT_GROUP&refreshEnabled=true
      ...
```

> 注：
>
> - 1、配置文件名要求为application.yml。
> - 2、客户端配置中的username与password层级变更至spring.cloud.nacos.discovery下。

> - 3、订阅配置文件的层级变更至spring.config.import，并推荐写法为“- optional:nacos:test1.yml?group=DEFAULT_GROUP&refreshEnabled=true”

2、spring-cloud-starter-hxnacc-config 1.0-RELEASE/1.1-RELEASE 或 spring-cloud-starter-nacos-config2021.1 bootstrap.yml 配置示例

```yaml
server:
  port: 8080
spring:
  application:
    name: test-server
  cloud:
    username: nacos  # 访问认证用户名，对应3.2.5章节中的“账号”
    password: nacos  # 访问认证密码，对应3.2.5章节中的“密码”
    config:
      namespace: TENANT-PROJECT-WORKSPACE  # 命名空间ID，用于隔离不同的环境或项目，对应3.2.5章节中的“命名空间”
      server-addr: *.*.*.*:18662  # 注册中心服务地址，多个地址用逗号分隔，对应3.2.5章节中的“部署地址”
      group: DEFAULT_GROUP  # 服务分组名称
      file-extension: yml # 配置文件类型
      shared-configs:
        - data-id: test1.yml
          refresh: true
        - data-id: test2.yml
          refresh: true
        ...
```

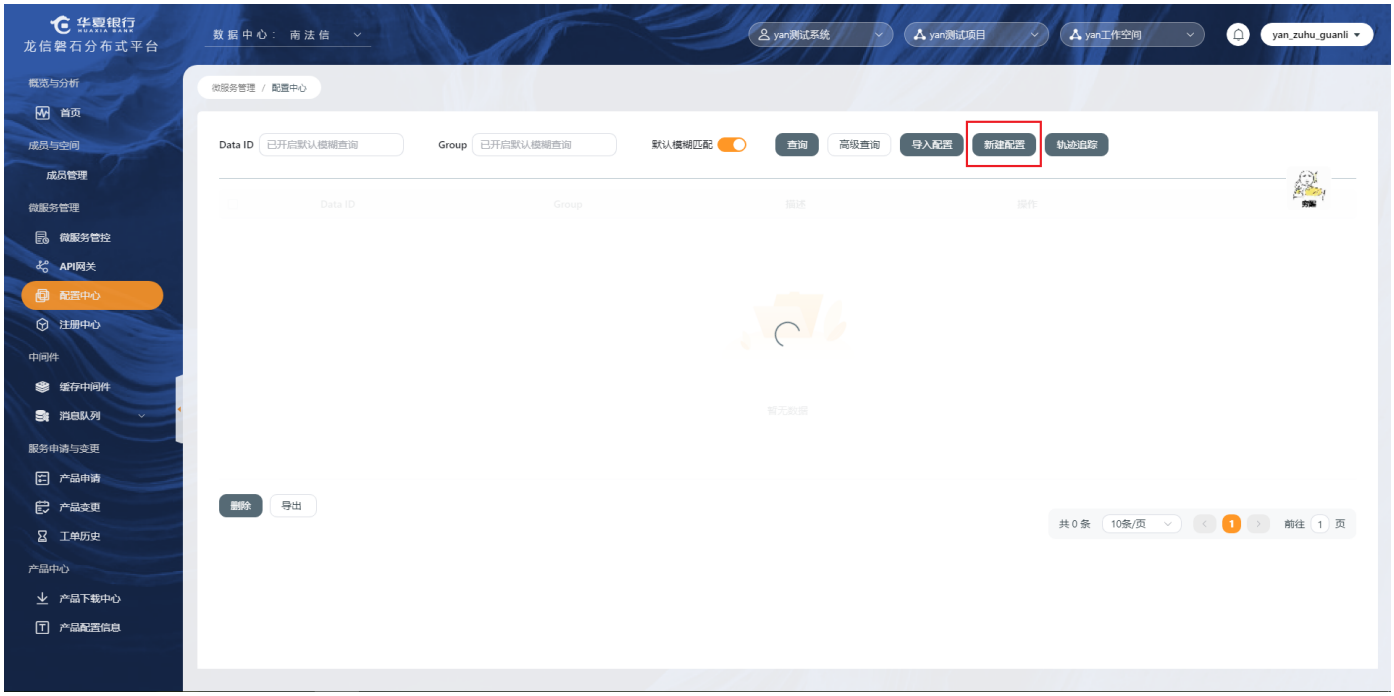
3.2.3 配置创建及导入

系统管理员登录磐石平台后，进入“微服务管理-配置中心”，在顶部栏选择所申请的数据中心、系统、项目和工作空间后新建或导入配置。

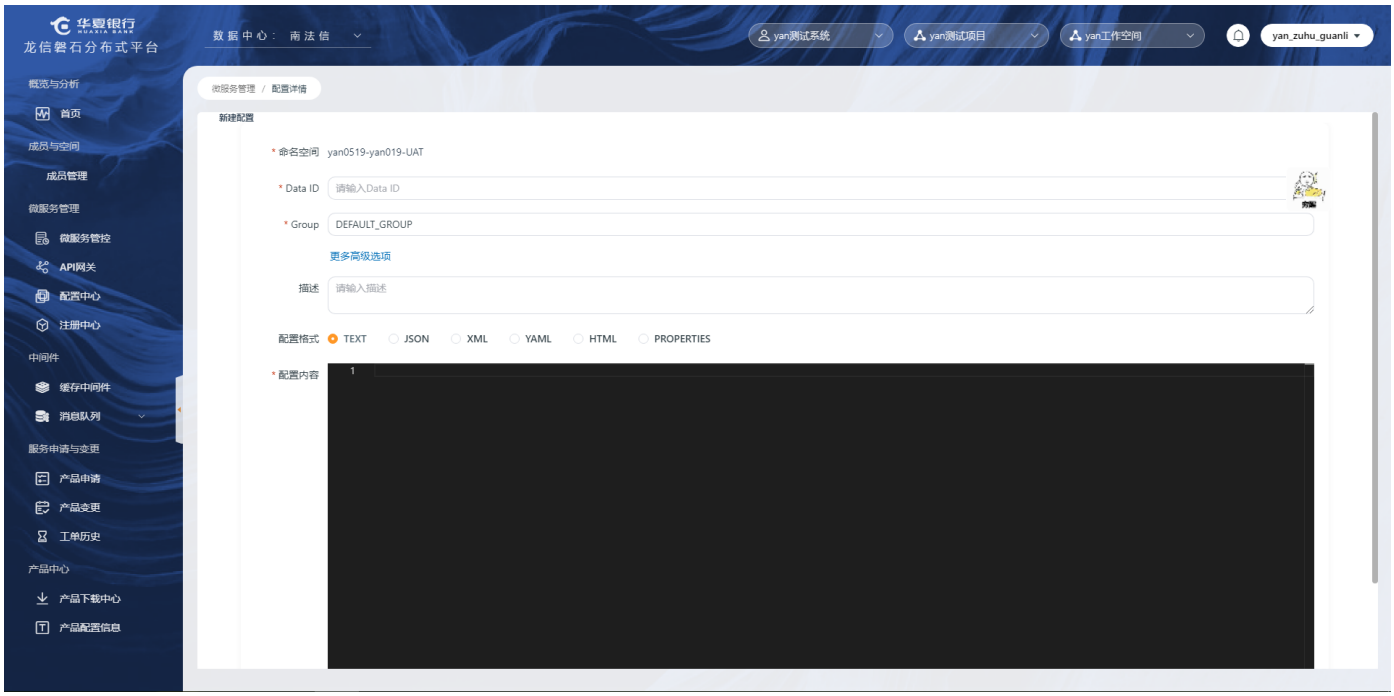


一、新建配置方式

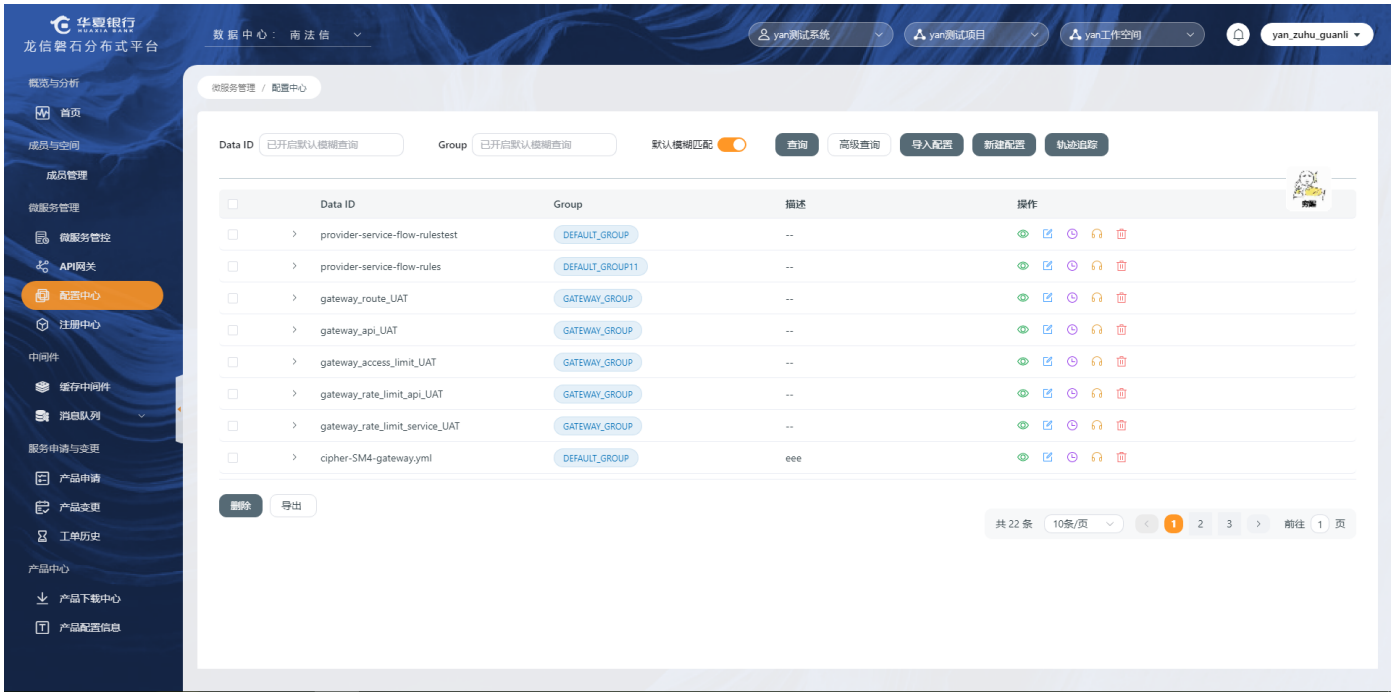
若采用新建配置方式，点击“新建配置”按钮，逐个新建配置文件。



完善配置文件信息及文件内容，点击“发布”。

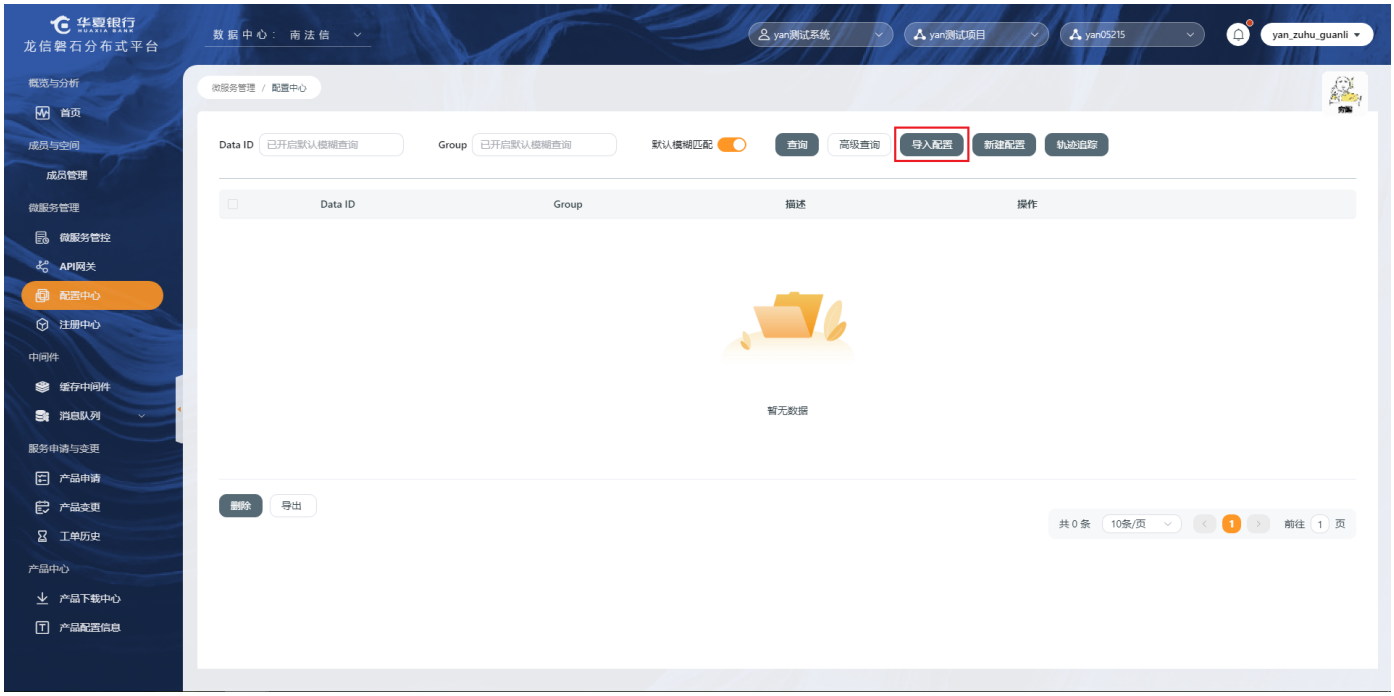


发布成功，可查看已新建的配置列表。

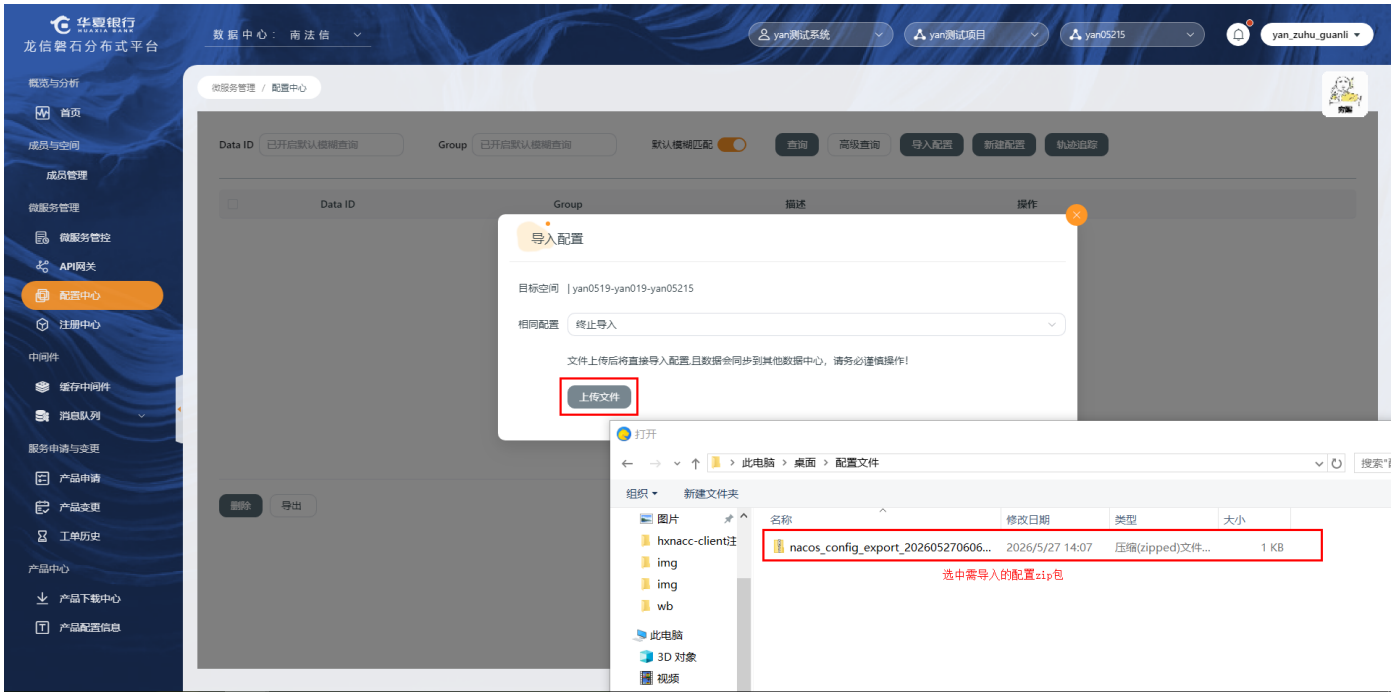


二、导入配置方式

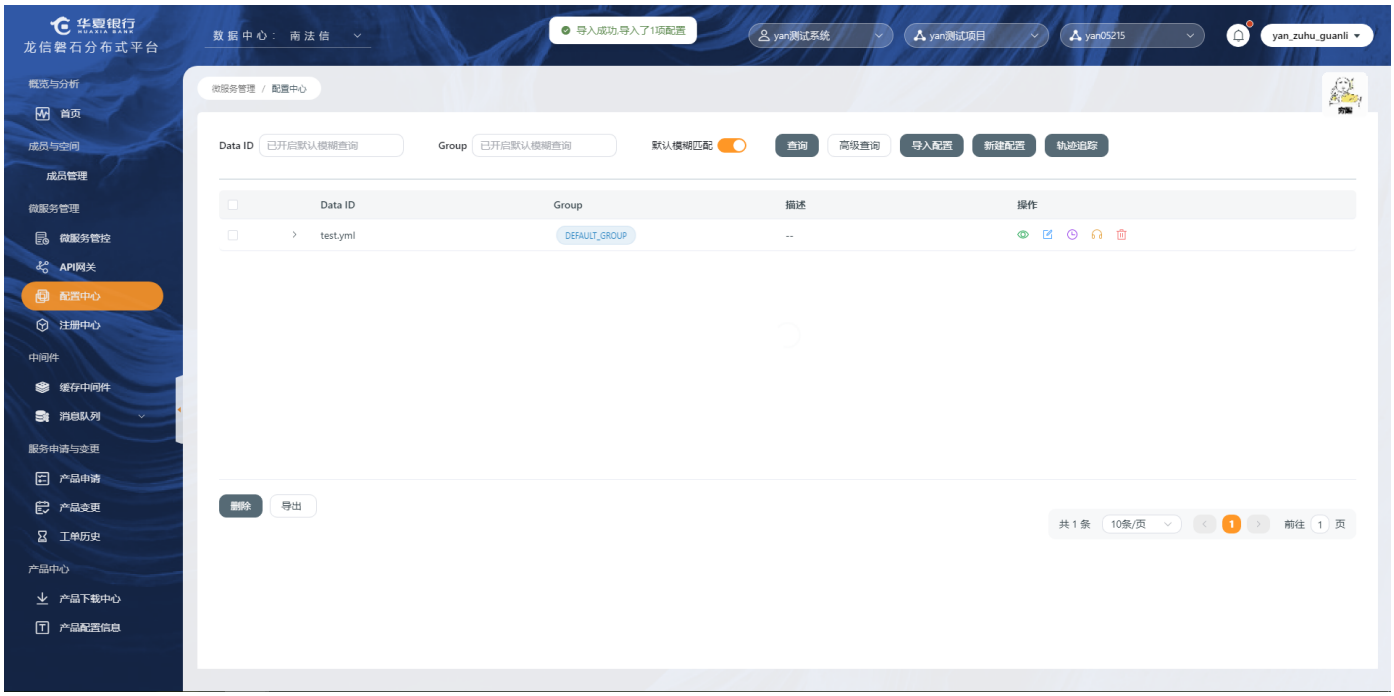
若采用导入配置方式，点击“导入配置”按钮。



选择需导入的配置zip包，点击打开。



导入成功，查看已导入的配置列表。



3.3 微服务管控

3.3.1 功能说明

- 服务列表与监控

  用于查看平台已接入的微服务和对应微服务所有实例，以及微服务中的各种管控规则信息。

- 流量控制（限流/熔断）

  用于保障系统在高并发场景下稳定运行。

- 服务治理策略

  平台提供统一的治理能力。

3.3.2 接入配置

- 前置条件：完成配置中心及注册中心产品申请
- 组件依赖
  - 需集成注册、配置中心相关SDK

```
<dependency>
    <groupId>com.hxb.cadp</groupId>
    <artifactId>spring-cloud-starter-hxnacc-discovery</artifactId>
    <version>***</version>
</dependency>
<dependency>
    <groupId>com.hxb.cadp</groupId>
    <artifactId>spring-cloud-starter-hxnacc-config</artifactId>
    <version>***</version>
</dependency>
```

  - 再接入磐石微服务管控SDK

```
<dependency>
    <groupId>com.hxb.cadp</groupId>
    <artifactId>spring-cloud-starter-hxsntl-autoconfig</artifactId>
     <version>1.1-RELEASE</version>
</dependency>
```

  - 应用配置

```
spring:
  cloud:
    nacos:
      config:
        namespace: PDCP-PDCP-DEMO
        server-addr: *.*.*.*:18662
        username: nacos
        password: nacos
        file-extension: yml
      discovery:
        namespace: PDCP-PDCP-DEMO
        server-addr: *.*.*.*:18662
        username: nacos
        password: nacos
```

    - 应用配置说明：
      - **微服务管控SDK，会根据配置文件中的注册、配置中心的用户名、密码等信息，自动接入磐石平台的微服务管控模块。**

- **注册、配置中心产品开通，以及用户名、密码等信息的获取可见本文档《3.1 注册中心》《3.2 配置中心》章节。**

> **注意事项：**
>
> 当前微服务管控客户端`hxsntl-autoconfig`的`1.1-RELEASE`版本支持的Spring Boot版本是`2.x.x`，还未适配Spring Boot版本`3.x.x`系列。

3.3.3 规则配置

登陆磐石平台，打开微服务管理>微服务管控

- 限流规则
  - 用于控制系统流量。

  配置项：



配置项	说明
资源名称	需要进行流量控制的接口、方法等
针对来源	仅针对特定微服务，默认针对所有微服务进行限流
阈值类型	可配置QPS或者线程数
阈值	每秒允许的最大通过请求数

- 熔断规则
  - 用于避免故障扩散

    配置项：



配置项	说明
资源名称	需要进行熔断控制的接口、方法等
熔断策略	针对服务调用的异常数、异常比例来触发熔断
阈值	触发熔断的阈值控制
熔断时长	触发熔断之后，熔断的时长

- 降级规则

- 用于保障核心业务可用，在服务发生熔断之后，响应服务繁忙信息。

- 热点参数规则
  - 用于保护热点资源

    配置项：



配置项	说明
资源名称	需要进行热点参数控制的接口、方法等
参数索引	需要控制的热点参数在接口的位置
流控模式	QPS模式
阈值	触发热点限流的阈值控制

- 系统保护规则
  - 用于系统自适应保护。

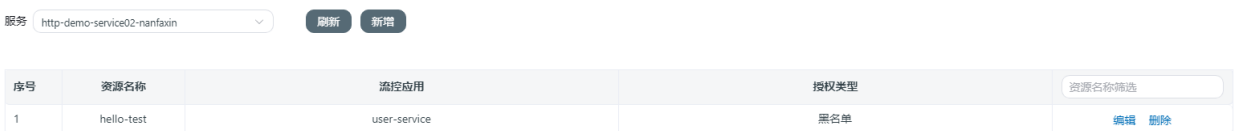
    配置项



配置项	说明
阈值类型	需要进行系统保护的系统资源类型
单机阈值	触发系统保护的阈值控制

- 授权规则
  - 用于保护系统

    配置项



配置项	说明
资源名称	需要进行授权控制的接口、方法等
流控应用	需要授权的微服务服务名
授权类型	微服务被纳入黑名单还是白名单

3.3.4 高阶使用

- 集群流控配置
  - 集群流控用于控制整个微服务集群的整体流量，例如限制某个API的总QPS。
  - 集群流控，需要微服务实例数大于等于2个，此时可以选择一个作为集群的流量统计服务器，专门用于统计系统总的调用量，其他的微服务实例都与**Token Server**通信来判断某个接口是否被限流。通过`新增 Token Server`之后，就可以使用集群流控的功能。



配置项：



配置项	说明
资源名称	需要进行授权控制的接口、方法等
是否集群	勾选之后，就启用集群流控
失败退化	若集群流控不可用，流控规则按照单机限流生效；若不勾选，默认不限流
集群阈值模式	集群的流量是根据限流阈值均摊到每一个实例还是整体阈值控制。

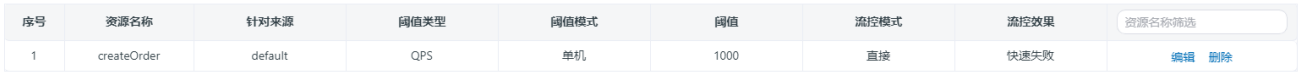
3.3.5 实践示例

- 限流

  场景：秒杀活动

  接口：`/create/order`

  配置：QPS为1000，流控效果是快速失败



序号	资源名称	针对来源	阈值类型	阈值模式	阈值	流控模式	流控效果	资源名称筛选
1	createOrder	default	QPS	单机	1000	直接	快速失败	编辑 删除

  - 当秒杀活动开始后，超出1000的请求会被立即拒绝。

  业务代码:

```
// OrderController
@PostMapping("/create/order")
public String createOrder() {
    orderDemoService.createOrder();
    return "订单创建成功";
}
```

```
// orderDemoService
@HxsntlResource(value = "createOrder")
public void createOrder() {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

  异常信息:

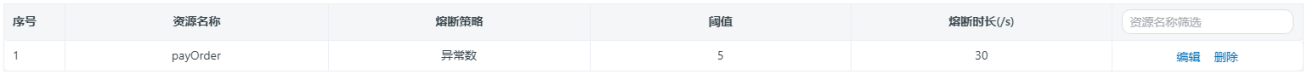
```
2026-05-26T14:51:29.337653800Hello run....
2026-05-26T14:51:30.231956200Hello run....
2026-05-26T14:51:31.014173900Hello run....
2026-05-26T14:51:32.290282200Hello run....
[ERROR][http-nio-9879-exec-7][DirectJDKLog.java][175][][2026-05-26 14:51:32,913]
[Servlet.service() for servlet [dispatcherServlet] in context with path [] threw
exception [Request processing failed; nested exception is
java.lang.reflect.UndeclaredThrowableException] with root cause]
com.hxb.cadp.hxsntl.slots.block.flow.FlowException: null
2026-05-26T14:51:33.536933200Hello run....
[ERROR][http-nio-9879-exec-2][DirectJDKLog.java][175][][2026-05-26 14:51:34,132]
[Servlet.service() for servlet [dispatcherServlet] in context with path [] threw
exception [Request processing failed; nested exception is
java.lang.reflect.UndeclaredThrowableException] with root cause]
com.hxb.cadp.hxsntl.slots.block.flow.FlowException: null
```

  - 在微服务的日志中，我们可以看见`FlowException`的流控异常，即触发了流量控制。

- 熔断

  场景：支付服务异常

  配置：支付异常数发生了5次之后，触发熔断，熔断时长30s



序号	资源名称	熔断策略	阈值	熔断时长(/s)	资源名称筛选
1	payOrder	异常数	5	30	编辑 删除

  业务代码

```
// OrderController
@PostMapping("/pay/order")
public String payOrder() {
    orderDemoService.payOrder();
    return "订单支付成功";
}
```

```
// orderDemoService
@HxsntlResource(value = "payOrder")
public void payOrder() {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

  异常信息：

  - 在业务异常达到配置的次数`5次`之后，就会触发熔断异常`DegradeException`，暂停访问支付服务，避免服务雪崩。

- 热点参数

  场景：热门订单商品访问

  配置：热门商品，goods_id=10001，限制QPS为100



序号	资源名称	参数索引	流控模式	阈值	是否集群	例外项数目	资源名称筛选
1	getOrderGoodsById	0	QPS	100	否	0	编辑 删除

  业务代码

```
@PostMapping("/pay/order/{goodsId}")
public String getOrderGoodsById(@PathVariable("goodsId") String goodsId) {
    return orderDemoService.getOrderGoodsById(goodsId);
}
```

```
// orderDemoService
@HxsntlResource(value = "getOrderGoodsById")
public void getOrderGoodsById(String goodsId) {
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

异常信息：

- 在查询商品`goods_id=10001`信息达到限流阈值100之后，就会触发热点参数限流异常`ParamFlowException`，保护数据库，避免热点击穿。

- 系统保护

  场景：大型促销活动

  配置：设置CPU阈值为80%

序号	阈值类型	单机阈值	操作
1	CPU 使用率	0.8	编辑 删除

  异常信息：

  - 当微服务的CPU负载超过80%，触发异常`SystemBlockException`，自动触发系统保护，拒绝部分流量。

3.4 API 网关

3.4.1 功能说明

- API网关管理功能主要包含以下部分:
  - API列表：API接口的统一管理，支持接口的增删改查；防重放、身份认证、优先级、是否公开访问、状态控制、超时时间等功能。经过网关的接口需要在此维护，维护完点击刷新生效，无需重启。
  - 访问控制：根据IP、自定义Header、自定义Param信息设置接口级别的访问控制,如黑白名单的访问控制。
  - 流量控制：根据API、IP、会话、自定义Header、自定义请求信息设置接口级别或者服务级别的流量控制。
  - 网关路由：网关路由的统一管理，支持路由的增删改查；可配置路由前缀、路由方式、失败重试、是否禁用等功能；以及路由手动刷新功能。
  - API网关也支持防xss攻击、防sql注入、鉴权放行/忽略等功能操作；

3.4.2 接入配置

- 前置条件：完成API网关产品申请

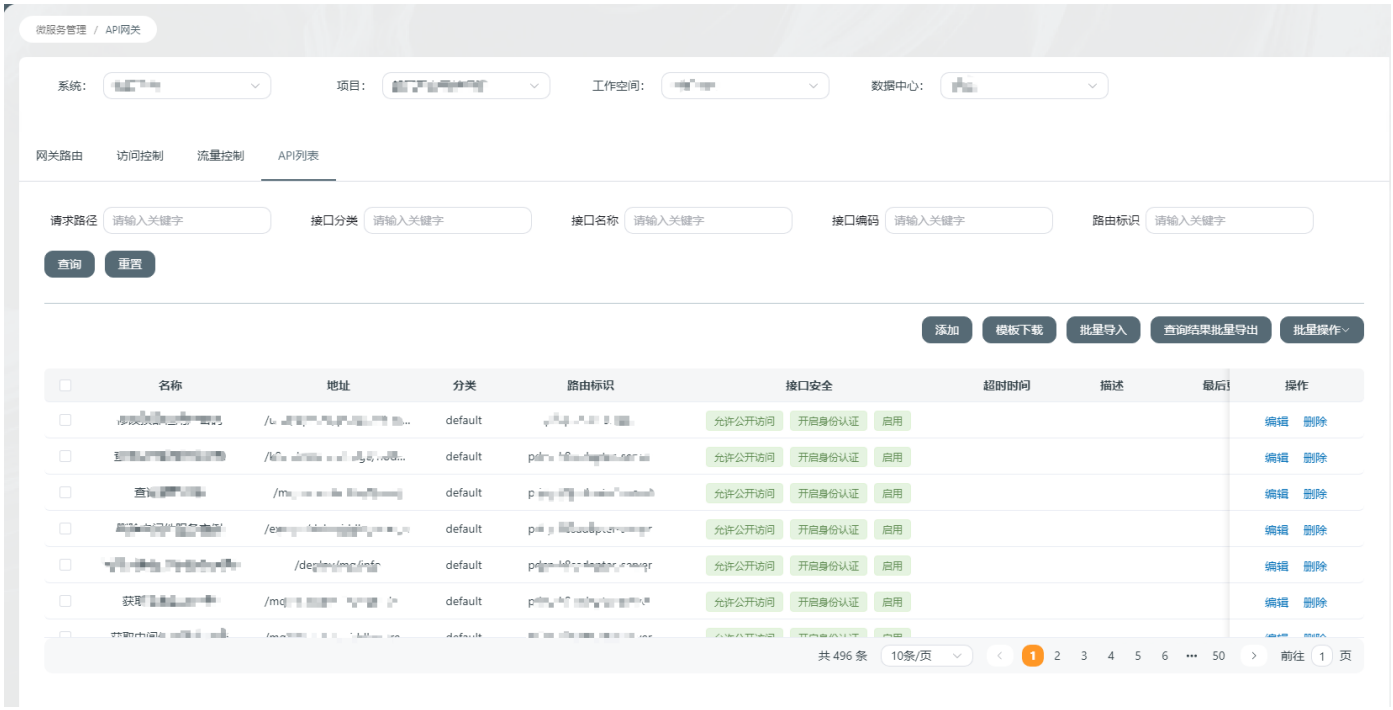
3.4.3 API 网关管理（配置新增及导入）

- API列表

（1）API列表查看搜索：

API列表统一管理所有应用的接口配置。所有的接口都应该在此添加并进行维护。

列表展示页可以根据请求路径、接口名称、接口编码、路由标识等进行查询搜索接口。



（2）API接口的添加和编辑



路由标识：此名称下拉列表来源自API列表-网关路由中配置的路由标识数据。

接口分类：自定义分类名称即可。

接口编码：系统自动生成。

接口名称：接口功能的中文描述。

是否验证防重放： 针对交易类的接口，为防止用户重复提交，防重放应开启验证。**示例请跳转 （5）防重放使用步骤示例**

请求地址： 填写接口的请求地址即可。

身份认证：租户使用身份验证需要先调用登录接口获取认证token和刷新token，然后在请求头携带。**示例请跳转（6）身份验证使用步骤示例**

公开访问：微服务间的接口，在添加接口的时候，公开访问应设置成拒绝。

状态：选择禁用：请求会提示"请求地址,禁止访问!" ，选择维护中：请求会提示"正在升级维护中,请稍后再试!"

超时时间： 网关到微服务接口默认超时时间为45S，也可通过配置接口超时时间个性化设置。

注：

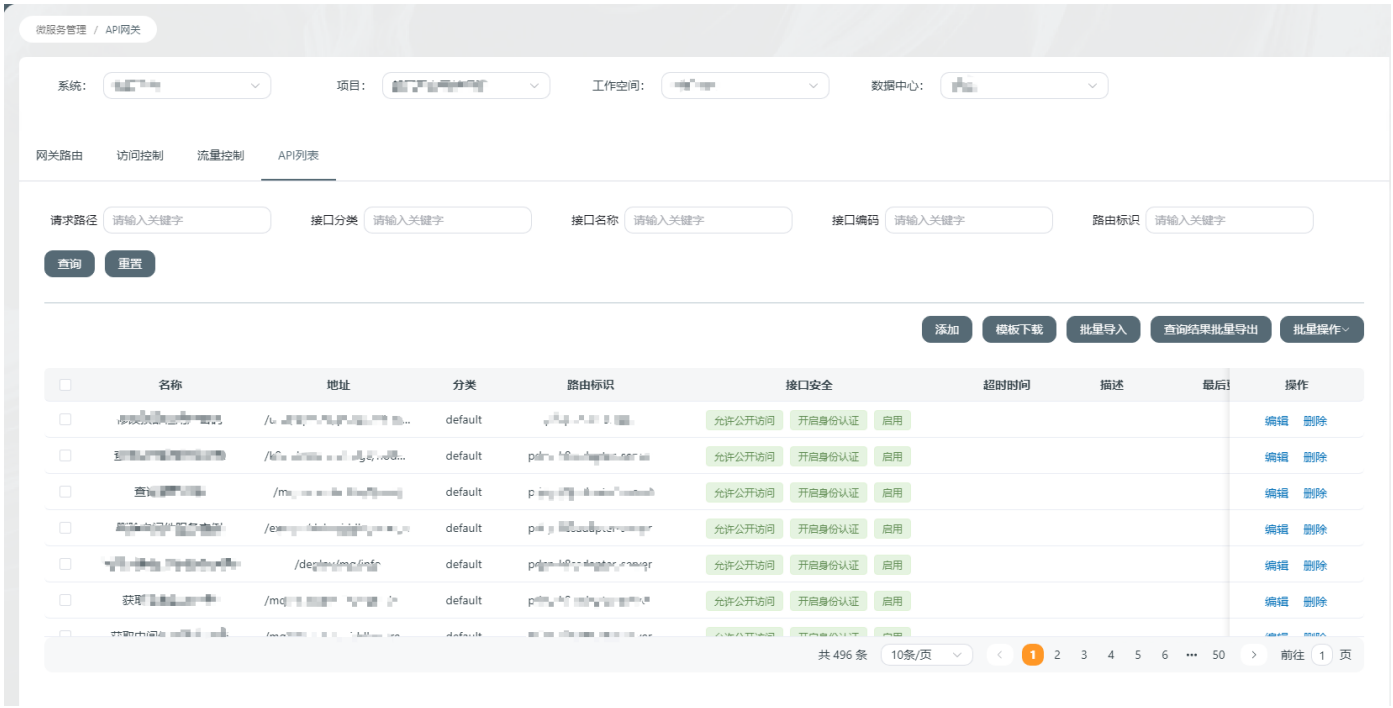
新增或者修改api时，可以手动点击网关路由页面的手动刷新网关按钮。

（3）API列表批量导入或导出：

点击"模板下载"，可以下载批量导入API接口的模板文件。

点击"批量导入"，选择文件点击确认批量导入API接口。

点击"查询结果批量导出"，导出当前查询结果的所有API列表到表格。



（4） 模板下载：下载的模板文件,需按照Sheet2页模板说明规则添加。

	A	B
1		模版字段说明
2	1	接口名称：接口功能的中文描述。
3	2	接口分类：自定义分类名称即可，默认请写default。
4	3	资源描述：接口的中文描述
5	4	路由标识：此名称下拉列表来源自API列表-网关路由中配置的服务列表数据中的路由标识。
6	5	请求路径：接口的请求地址，以“/”开头
7	6	优先级：优先级越小越靠前。
8	7	状态:0-禁用 1-启用 2-维护中
9	8	是否需要认证：0-无认证 1-身份认证
10	9	是否公开： 0-内部的 1-公开的
11	10	验证防重放（0.不验证 1.验证）
12	11	请求方式：api接口的请求方式
13	12	响应类型：api接口的响应类型
14	13	类名：api接口所属的类的名字
15	14	方法名：api接口所属的方法的名字
16	15	超时时间：网关到微服务接口默认超时时间为45S，如果使用默认超时时间，填0即可，也可以根据项目组自身需求配置超时时间。单位是毫秒
17	16	保留数据：0-允许删除 1-不允许删除 默认为0
18		其它说明
19	1	红色字段为必填字段。
20	2	表格sheet名称不能修改,上传时只会导入sheet名称为"API"的sheet页内数据。
21	3	首行(标题行)不能删除和修改。
22	4	导入长传只支持.xls和.xlsx后缀
23		
24		

（5）防重放使用步骤示例

1. 初始流程步骤（baseUrl为租户网关暴露的ip和端口）
   首先调用 http://{{baseUrl}}/token/geSessionId 接口获取sessionId ，
   响应结果如下

```
{
    "returnCode": "000000",
    "message": "success",
    "path": null,
    "data": "a8a8dac91afb4cb1b1a50d23a0677f6f", //返回的是sessionId
    "extra": null,
    "timestamp": 1779760267208
}
```

2.然后调用http://{{baseUrl}}/token/getToken 接口获取rev 请求头需要携带sessionId

```
sessionId:a8a8dac91afb4cb1b1a50d23a0677f6f
```

响应结果如下：

```
{
    "returnCode": "000000",
    "message": "success",
    "path": null,
    "data": [
        "481b8a67ae8c4fb1a087c0bbd3f0521b", ////返回的是rev
        "aecb191ca6ff41ddb6abc23a320f8d7a",
        "9d0960a408f34d8e97970e5b80777fb5",
        "2c36276d3599409ba043c4a6307b2b17",
        "323cec29e50d4acf991e251048665be2",
        "bcc83bfcae734d1a897d54c904e5fac1",
        "d14d23ae56354defba9b1aa618528efd",
        "f2d70fc7e9d44ca5a41aa7a15ffb2837",
        "7d3625f51a8a459e91f3376065ee7921",
        "04921ce6a3fd49fcbb269a236cef5603"
    ],
    "extra": null,
    "timestamp": 1779759829836
}
```

3、在需要防重放保护的接口请求中携带两个token作为请求头入参：

```
rev:481b8a67ae8c4fb1a087c0bbd3f0521b
OAUTH2SESSION:a8a8dac91afb4cb1b1a50d23a0677f6f
```

（6）**身份验证使用步骤示例**

调用磐石平台登录接口获取认证token和刷新token

http://{{baseUrl}}/platform/login/token

```
{
    "username": "xxx",
    "password": "xxx",
    "captcha": "x",
    "lastRev": "xxx",
    "loginFlag": "0"
}
```

示例Authorization的值应为认证token，Cookie携带刷新token

```
Authorization: Bearer
eyJraWQiOiIzOTQzYTNjMC05ZTk1LTRlNDEtYTJjMy01OTMxODUwOTJjYjEiLCJhbGciOiJSUzI1NiJ9.eyJzdWIiO
iJ0ZXN0bGptIiwiYXVkIjoicGFuc2hpLWNsaWVudCIsIm5iZiI6MTc3OTc2MDc5MCwic2NvcGUiOlsicmVhZCJdLCJ
yb2xlcyI6W3sicm9sZUlkIjoyLCJyb2xlQ29kZSI6IlBEQ1BfQURNSU4iLCJyb2xlTmFtZSI6IuW5s-
WPsOeuoeeQhuWRmCJ9XSwiaXNzIjoiaHR0cDovLzEwMy4xNjAuNTUuODU6ODA4OCIsImV4cCI6MTc3OTc2MjU5MCwi
dXNlck5hbWUiOiJ0ZXN0bGptIiwiaWF0IjoxNzc5NzYwNzkwLCJ1c2VySWQiOjIwNTY2NjM5OTc1MTEzNzI4MDEsIm
p0aSI6Ijc4OTkwMGI5LTIxYTItNGFkMi04ZThlLWFkNTMyY2E1ZjU4NiJ9.qvrMiMwDEO0P6itJZHfMVBwJcqgze3y
pW9CF7c-YgEtpxJMHXf3IYFamtfHEBNdq5ED_XtilSPun7GNNg2HiklqCFZEd0bbyLyWVuS_eto1-
U3fJDq5rMx1MeVgyh-XsWs_T9cXvagCKW8m67v8iAUQ34XgsAWn7iV4x2cqia91f-
pD3k3fKSc22GLLram4B0ry_s_0niJGVLY2G7tuyn5evJ6907H0NMyz6Xxjw39A6dAJb2bcC0fwZA1Ix4e_07cbn9OB
SbVKBsFpgGDhtzg298CKW8JoR5a-HDk9qUB5zC_MwKpHK3sDiBdclUPWfMcHfNJRNMjrm377SBG0Slg
Cookie:
refresh_token=aaJK5PsE2MR9KAShfPiFKKkmBhjmSzFvmqjtzptsin0avZEv5EQx1jSFbQ4TXEA1LnaNMBK2y95P
Bf5luDT7mIMWn-33gie8aQilWP7hSd_R2Mm3DzroXJKAueYlwoCR
```

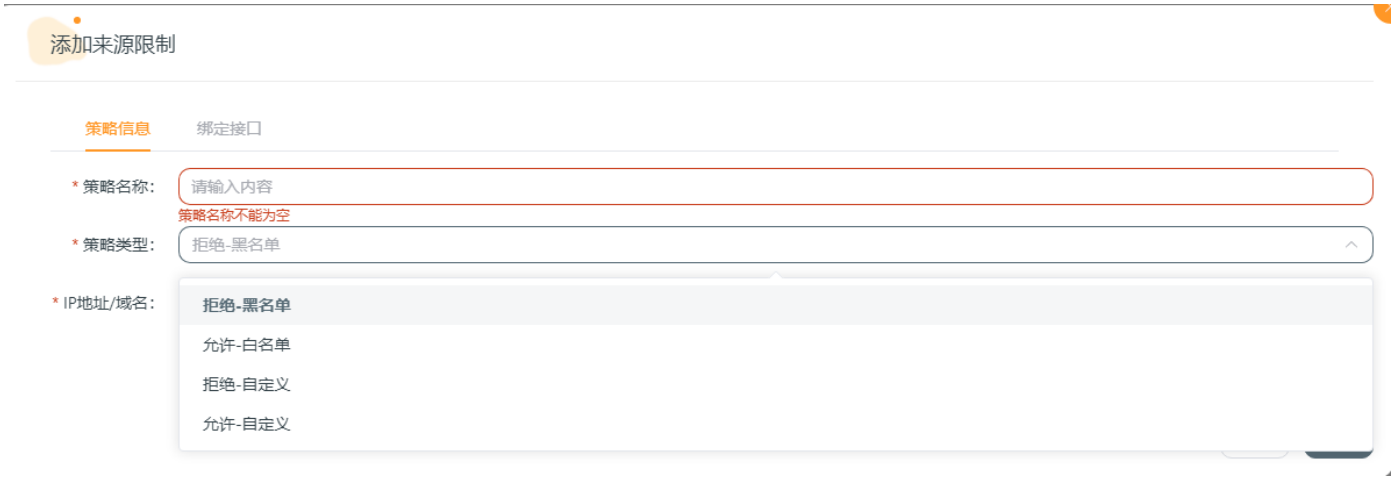
- 访问控制

  （1）访问控制页

  访问控制的统一管理，支持访问策略的增删改查，关联对应的接口。



  （2）添加访问控制策略



a. 策略类型分为两大类：拒绝、允许。

b. 拒绝黑名单、允许白名单：可以配置拒绝或者允许访问的IP地址/域名；同时支持Ip和域名,多个用分号";"隔开。示例：127.0.0.1;baidu.com;weixin.com 。

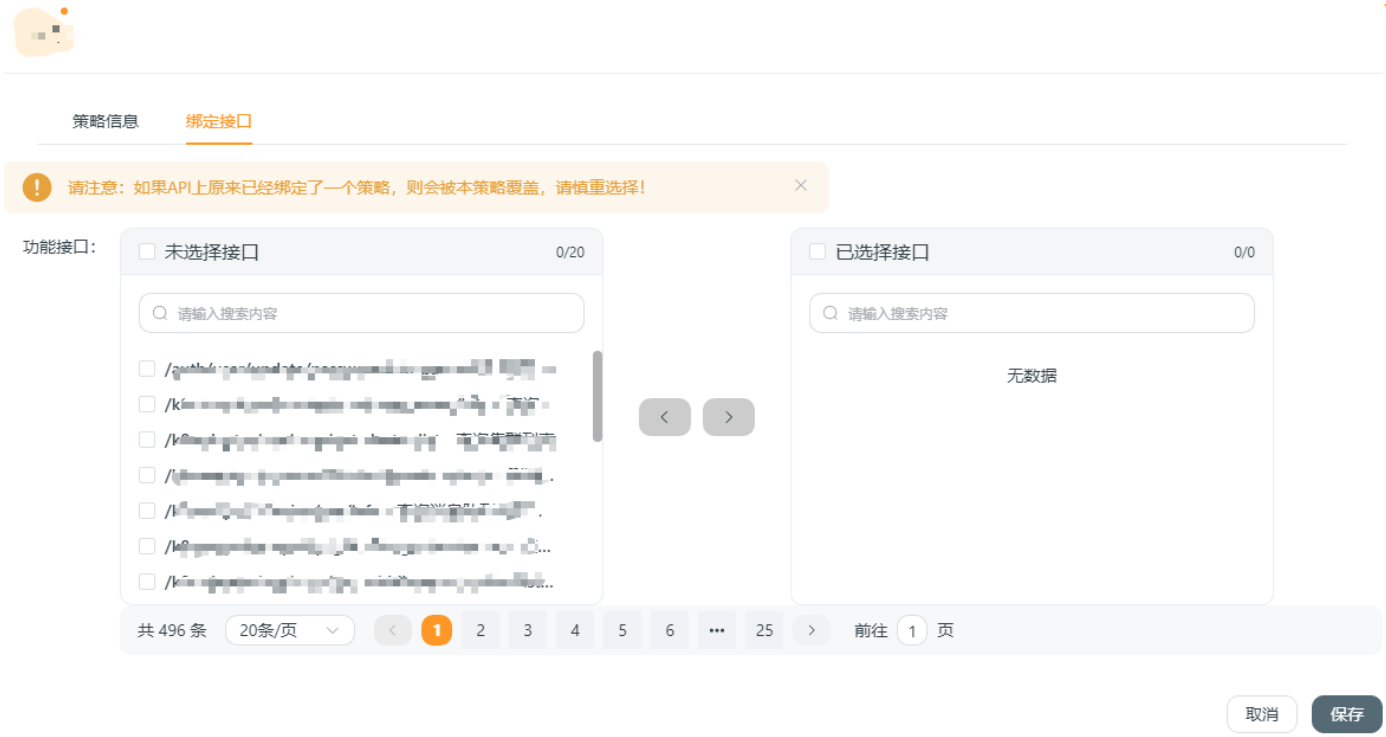
d. 拒绝自定义 、允许自定义：可以根据请求参数或者Header中信息进行访问控制。

格式：key,value ， 多个用分号";"隔开。（示例：userId,13120460000 或 merchantId,1005;appId,00001）



选择策略，绑定接口

一个策略支持绑定多个接口。如果一个接口被多个策略绑定，只会生效一条，后面的策略覆盖前面的策略。



- 流量控制

（1）流量控制列表页：支持流量控制策略的增删改查，关联绑定对应的接口，或者关联绑定微服务。



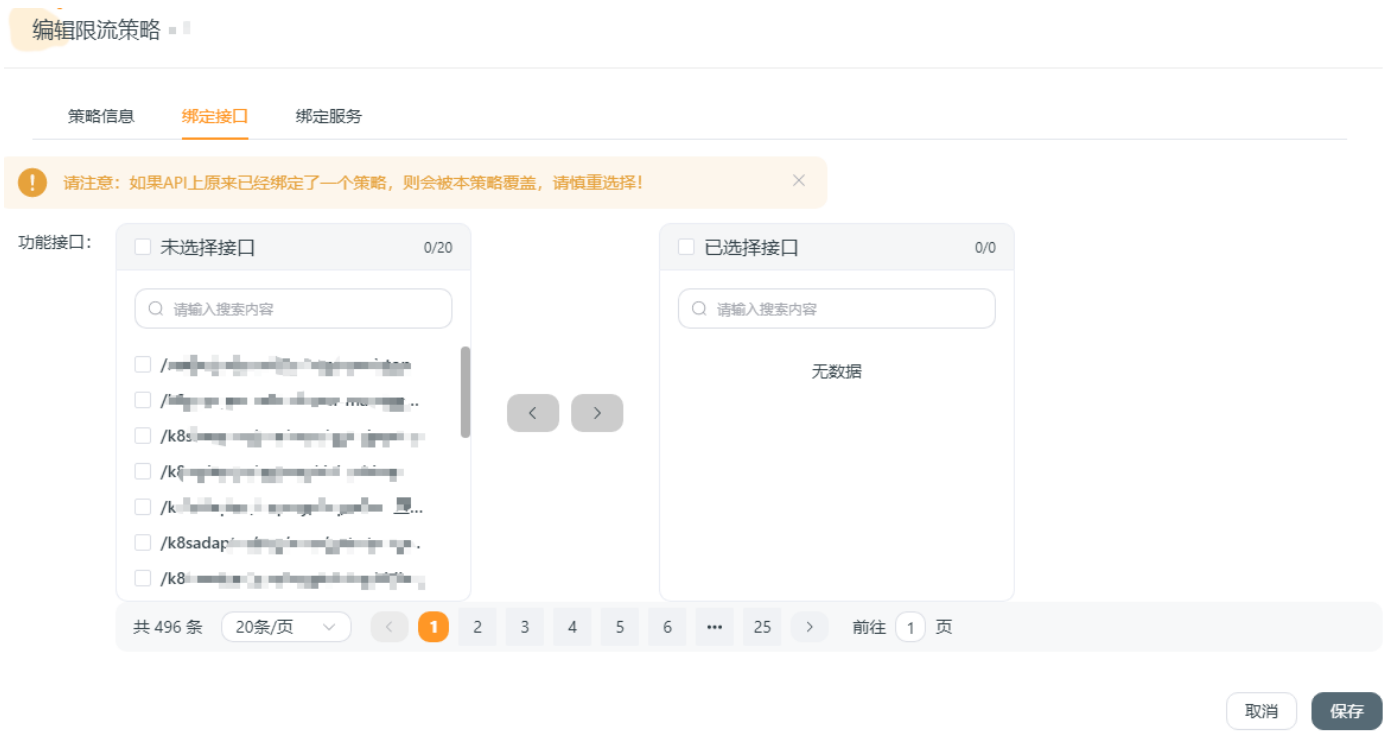
（2）添加限流策略

单位时间: 秒。

限流策略可分为5个维度：API流量控制、IP流量控制、会话流量控制、Header自定义流量控制、请求自定义流量控制。一个策略设置 中可以配置多个维度的限流，都会生效，只要触发其中一个维度的限流次数就会触发限流。

策略配置后实际生效的规则是按照秒来进行控制的。

限流采用的是令牌桶算法。

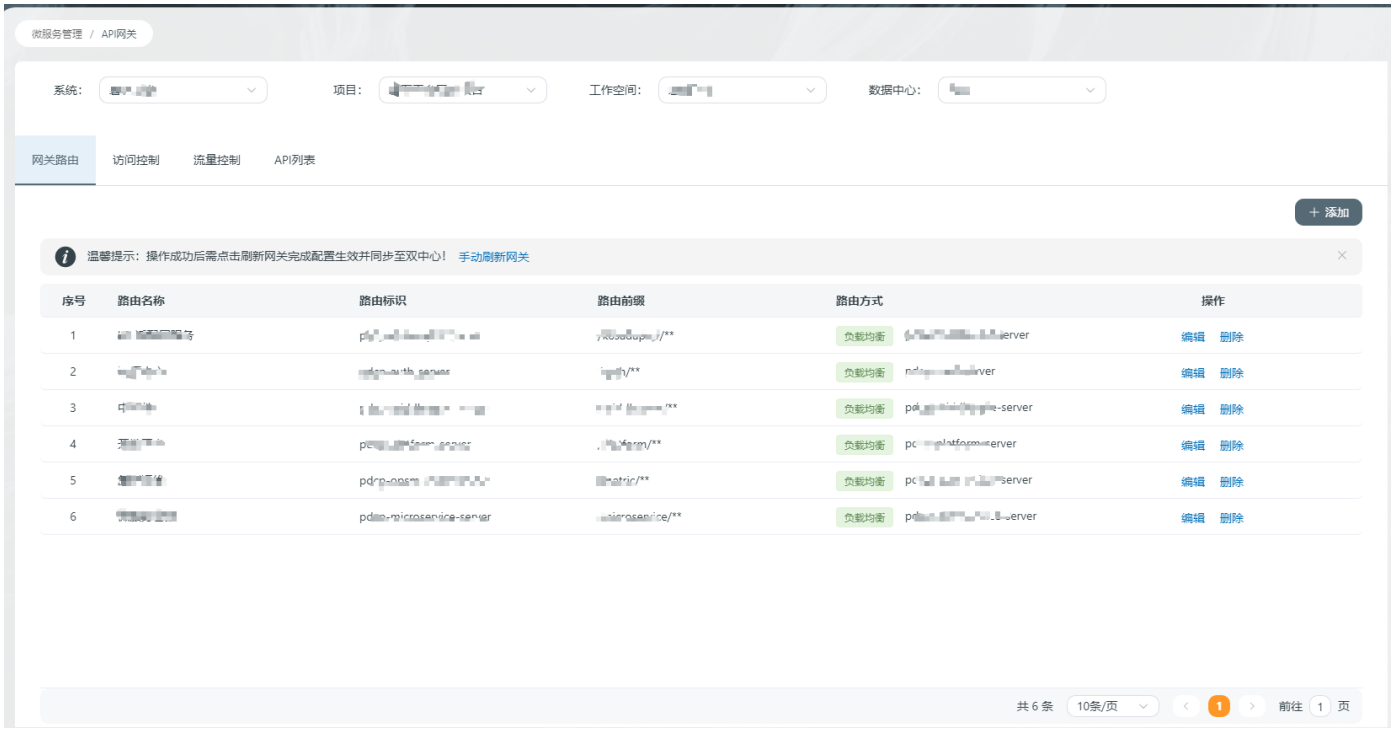


选择策略，绑定服务



- 网关路由

  (1) 网关路由列表页：支持路由的增删改查，以及路由手动刷新功能。



(2) 添加路由



a. 网关路由支持两种方式：负载均衡和反向代理。

b. 保留前缀：指的是网关转发到微服务的时候会继续携带路由前缀。参考如下示例说明：

> 假如地址 http://103.160.88.88:8888
> 页面请求↓
> http://103.160.88.88:8888/pmobile/base/get/api
> 经过nginx代理,请求到网关：http://网关ip:端口/base/get/apid
> 网关路由根据 /base 匹配到base服务,从nacos获取base服务实例地址
> 将请求转发到服务
> 如果保留前缀是：http://base服务ip:端口/base/get/api
> 如果保留不保留前缀是：http://base服务ip:端口/get/api

c. 负载均衡：填写服务名称，不要出现下划线“_”。

d. 手动刷新网关：点击后会刷新网关加载的数据资源，包括防重放攻击api接口列表、应用加密信息、超时时间接口列表、权限、访问控制、限流控制、加密接口列表、路由信息等数据。

3.5 缓存

3.5.1 功能说明

- 支持的缓存模式

  三种：单机模式（Standalone）、哨兵模式（Sentinel）、集群模式（Redis Cluster）。

- Redis 集群架构

  1. 单机模式（Standalone）

     仅运行一个 Redis 实例，不提供高可用或数据冗余。适合开发、测试或对可用性要求较低的场景。

  2. 哨兵模式（Sentinel）

     在主从复制的基础上增加了哨兵进程，哨兵通过心跳机制持续监控 Redis 主节点和从节点的状态。当发现主节点宕机时，多个哨兵节点会通过投票达成共识，自动将一个健康的从节点提升为新的主节点，并通知客户端更新连接地址。适用于数据量在单机内存承受范围内，但对服务可用性（HA）要求极高的生产环境。

     一个主节点、若干个从节点（建议≥1）。以及至少 3 个哨兵节点。目前可选的哨兵节点数为3，5，7，9。

  3. 集群模式（Redis Cluster）

     该模式采用去中心化的多主多从架构，通过哈希槽（Hash Slot）机制将键空间划分为 16384 个槽，并把数据自动分散存储在多个主节点上，从而实现了存储容量的水平扩展。同时，每个主节点都配有从节点，具备完善的自动故障转移能力。适用于需要分布式存储和水平扩容得场景。

     由多个 Redis 节点组成，通常包含至少 3 个主节点和对应的从节点（如 3 主 3 从）。目前可选分片数为3，5，7，9，且每个分片默认为1主1从。

3.5.2 接入配置

- 前置条件：完成缓存产品申请
- 连接地址获取

  1.需要根据2.1章节完成业务系统平台使用申请。

2.申请流程完成后，访问磐石系统，查看产品中心-产品配置信息，查看缓存中间件，点击账号信息,获取连接地址。



- 组件依赖

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
    <version>xxxx</version>
</dependency>
```

> 注：客户端版本建议
>
> jdk8：spring-boot-starter-data-redis >= 2.6.15 （Lettuce >= 6.1.10）
>
> jdk17：spring-boot-starter-data-redis >= 3.0.x （Lettuce >= 6.2.1）

- 连接配置

  以spring-boot工程配置示例：

  1. 单机模式（Standalone）

```
spring:
  data:
    redis:
      host: 127.0.0.1
      port: 6379
      username: your-username
      password: your-password
      database: 0
      timeout: 2000ms
      lettuce:
        pool:
          max-active: 20
          max-idle: 10
          min-idle: 5
```

  2. 哨兵模式（Sentinel）

```yaml
spring:
  data:
    redis:
      sentinel:
        master: mymaster
        nodes:
          - 127.0.0.1:26379
          - 127.0.0.1:26380
          - 127.0.0.1:26381
        username: your-username
        password: your-password
      username: your-username
      password: your-password
        database: 0
      timeout: 2000ms
      lettuce:
        pool:
          max-active: 20
          max-idle: 10
          min-idle: 5
```

3. 集群模式（Redis Cluster）

```yaml
spring:
  data:
    redis:
      cluster:
        nodes:
          - 127.0.0.1:7000
          - 127.0.0.1:7001
          - 127.0.0.1:7002
          - 127.0.0.1:7003
          - 127.0.0.1:7004
          - 127.0.0.1:7005
        username: your-username
        password: your-password
        max-redirects: 3
      timeout: 2000ms
      lettuce:
        pool:
          max-active: 20
          max-idle: 10
          min-idle: 5
```

3.5.3 高阶使用

- 关键配置说明

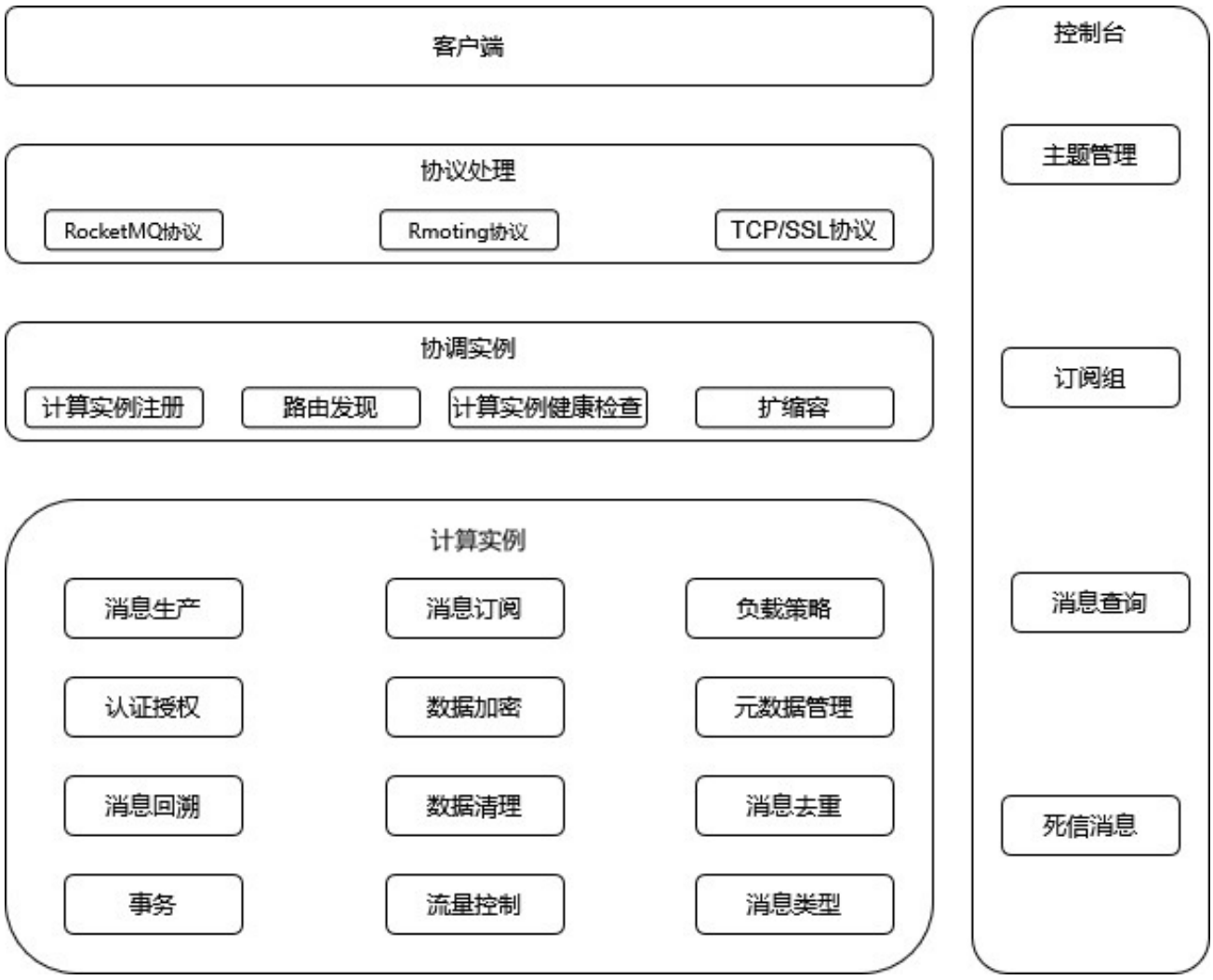
  目前缓存集群支持如下部署参数的自定义：项目组根据自身情况，可在申请集群时说明配置需求。

参数名	默认值	描述
maxMemoryPercentOfLimit	0	最大内存驱逐坚持度,取值范围0~100的整数（百分比）;0,代表没有限制;（只对CLUSTER集群模式生效）
maxmemory-policy	volatile-lru	内存淘汰策略
save	3600 1 300 100 60 10000	保存数据库到磁盘。如果经过指定的秒数并且对数据库的写操作超过了指定次数，Redis将保存数据库。格式： save <秒数> <变更次数> [<秒数> <变更次数> ...]
tcp-keepalive	60	TCP keepalive间隔，单位为秒。设为0可禁用该功能。默认值为60。
timeout	300	客户端空闲N秒后关闭连接（设为0可禁用）。默认值为300。

3.6 消息队列

3.6.1 功能说明

- 集群架构



CloudMQ-H是一款面向企业级场景设计的高性能、低延迟的分布式消息中间件，适用于各类大规模、低延时、高可靠的消息业务场景，以下从客户端、协议处理、协调实例、计算实例、控制台五个维度展开说明：

1. 客户端

支持同步/异步/单向消息发送、顺序消息、延迟消息、事务消息等高级特性，并内置负载均衡与容错机制，确保消息高可靠投递。

2. 协议处理

基于高效二进制协议与 Netty 异步非阻塞 IO 模型，CloudMQ-H显著降低网络传输开销，支持 TCP/SSL 通信，通过长连接复用与轻量级编解码设计，实现高吞吐、低延迟的集群通信，兼顾跨云混合部署场景的安全性与兼容性。特别的，产品已完美兼容RocketMQ协议，针对RocketMQ协议应用程序可零改造快速接入。

3. 计算实例（Broker）

计算实例基于 CommitLog 顺序写与索引文件分离的存储结构，结合主从同步（同步/异步复制）与数据刷盘机制，确保消息零丢失和高吞吐写入；通过零拷贝技术与冷热数据分离策略，实现海量消息堆 积与低成本存储。

4. 协调实例（Coordinator）

协调实例采用去中心化无状态架构，动态维护“主题”与“计算实例”的路由信息，实时感知计算实例上下线并自动更新路由表。客户端通过轻量级轮询机制获取路由，避免强依赖注册中心，保障集群高可用。其核心价值在于简化服务发现流程，支持计算实例动态扩缩容，降低运维复杂度，提升系统弹性与稳定性。

5. 控制台

   管理控制台提供可视化运维能力，涵盖主题、订阅组、消息查询、死信消息等管理，帮助运维管理人员快速搭建消息服务及简化日常运维操作。另外还有消息轨迹查询、死信处理等治理操作，帮助开发者快速定位问题，降低运维门槛。

- 消息类型
  - 普通消息：支持生产消费普通消息，提供同步、异步、单向发送模式，支持消息持久化存储和可靠投递。
  - 顺序消息：通过消息队列（Queue）的分区顺序性，可保证同一队列内消息按发送顺序被消费。
  - 定时/延时消息：消息发送时可设置固定延迟级别或精确时间戳，计算实例按时间触发投递。
  - 事务消息：通过两阶段提交（半事务消息+状态回查）实现分布式事务最终一致性。
  - 批量消息：生产者可一次性发送多条消息，减少网络开销，提升吞吐量。
- 消费模式
  - 拉取式消费（Pull Consumer）：Consumer消费的一种类型，应用通常主动调用Consumer的拉消息方法从Broker服务器拉消息、主动权由应用控制。一旦获取了批量消息，应用就会启动消费过程。
  - 推动式消费（Push Consumer）：Consumer消费的一种类型，该模式下Broker收到数据后会主动推送给消费端，该消费模式一般实时性较高。

3.6.2 接入配置

- 前置条件：完成消息队列产品申请
- 连接地址获取：

  1.需要根据2.1章节完成业务系统平台使用申请。

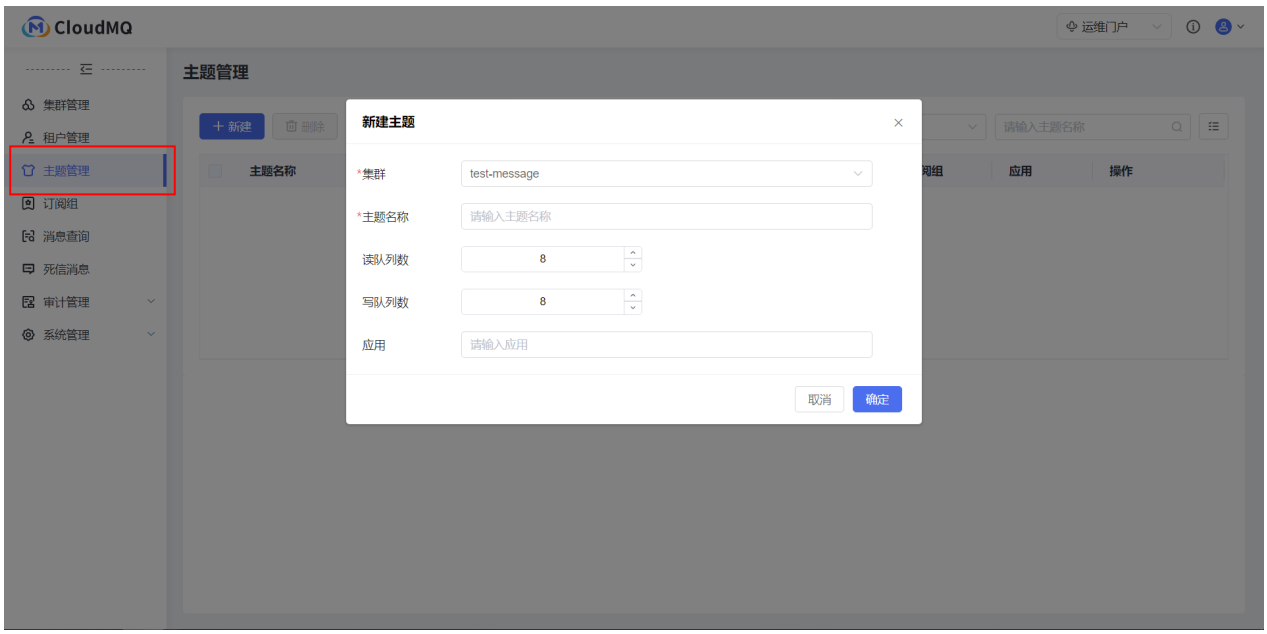
  2.申请流程完成后，访问磐石系统，查看产品设置，查看消息中间件，获取连接地址（客户端地址/账号/密码；管理端地址账号/密码）。

  如下图所示：

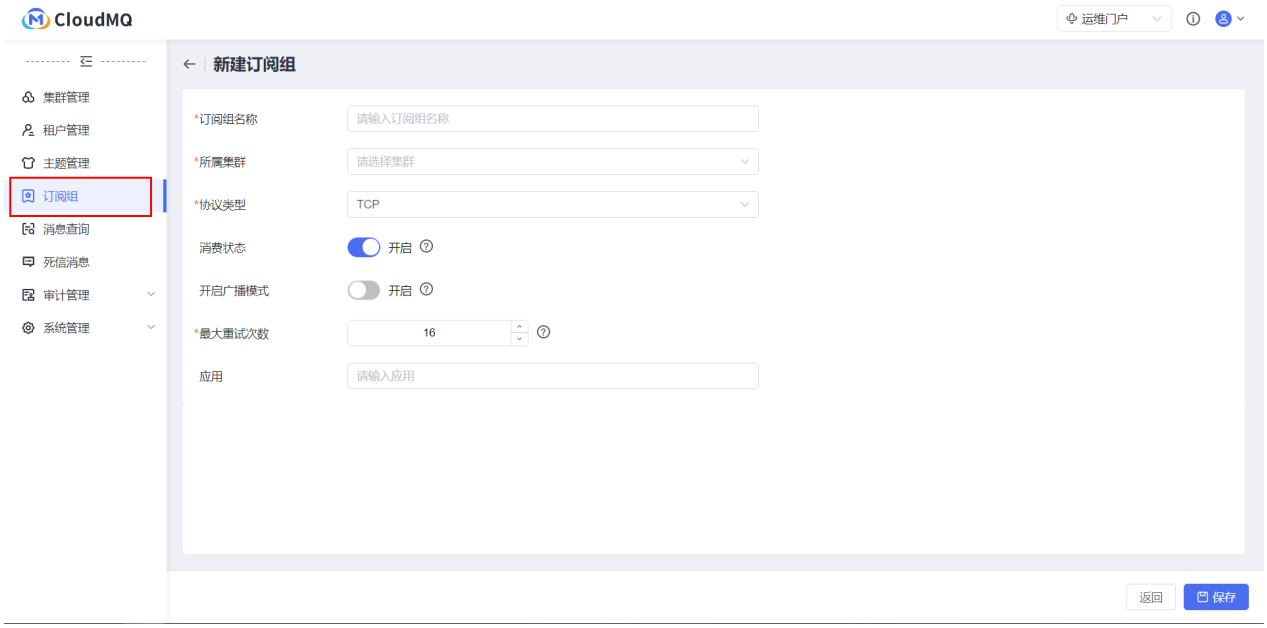


- 连接前的配置：需要通过管理端在要连接的集群下创建主题和订阅组

- 创建主题



- 创建订阅组



- 组件依赖

  bes-cloudmq-h-client-all-5.10.0.jar

```
<dependency>
    <groupId>com.bes.cloudmq.amphi</groupId>
    <artifactId>bes-cloudmq-h-client-all</artifactId>
    <version>5.10.0</version>
</dependency>
```

- 连接配置

  以下参数建议配置到启动脚本或YAML文件。

  - 生产者

        - 生产组名称
        - 协调地址
        - 主题
        - 客户端账号
        - 客户端密码
    - 消费者
        - 订阅组
        - 协调地址
        - 主题
        - 客户端账号
        - 客户端密码

```
# 配置项名称和层级关系按需自定义，以下只是示例
mq:
  coordinatorAddr: 127.0.0.1:8888
  username: username
  password: password
  topic: xxx
  producer:
    name: xxx
  consumer:
    subscribeGroup: xxx
```

3.6.3 高阶使用

- **3.6.3.1 顺序消息**
    - 发送顺序消息

```
public class Producer {
    public static void main(String[] args) throws Exception {
        DefaultMQProducer producer = new
            DefaultMQProducer("producer_group_name");
        producer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置生产者所属中心
        // producer.setAz("az1");
        producer.start();
        String[] tags = new String[]{"TagA", "TagC", "TagD"};
        // 订单列表
        List<OrderStep> orderList = new Producer().buildOrders();
        Date date = new Date();
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
        String dateStr = sdf.format(date);
        for (int i = 0; i < 10; i++) {
            // 加个时间前缀
            String body = dateStr + " Hello BES CloudMQ " + orderList.get(i);
```

```
            Message msg = new Message("TopicTest", tags[i % tags.length], "KEY"
                                      + i, body.getBytes());
            SendResult sendResult = producer.send(msg, new
                                                  MessageQueueSelector() {
                                                      @Override
                                                      public MessageQueue
select(List<MessageQueue> mqs, Message msg,

Object arg) {
                                                          Long id = (Long) arg;
//根据订单id选择发送queue
                                                          long index = id %
mqs.size();
                                                          return mqs.get((int)
index);
                                                      }
                                                  },
orderList.get(i).getOrderId());//订单id
            System.out.println(String.format("SendResult status:%s, queueId:%d,
                                             body:%s",
                                             sendResult.getSendStatus(),

sendResult.getMessageQueue().getQueueId(),
                                             body));
        }
        producer.shutdown();
    }
    /**
* 订单的步骤
*/
    private static class OrderStep {
        private long orderId;
        private String desc;
        public long getOrderId() {
            return orderId;
        }
        public void setOrderId(long orderId) {
            this.orderId = orderId;
        }
        public String getDesc() {
            return desc;
        }
        public void setDesc(String desc) {
            this.desc = desc;
        }
        @Override
        public String toString() {
            return "OrderStep{" +
                "orderId=" + orderId +
                ", desc='" + desc + '\'' +
                '}';
        }
```

```
    }
    /**
* 生成模拟订单数据
*/
    private List<OrderStep> buildOrders() {
        List<OrderStep> orderList = new ArrayList<OrderStep>();
        OrderStep orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111039L);
        orderDemo.setDesc("创建");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111065L);
        orderDemo.setDesc("创建");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111039L);
        orderDemo.setDesc("付款");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103117235L);
        orderDemo.setDesc("创建");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111065L);
        orderDemo.setDesc("付款");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103117235L);
        orderDemo.setDesc("付款");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111065L);
        orderDemo.setDesc("完成");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111039L);
        orderDemo.setDesc("推送");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103117235L);
        orderDemo.setDesc("完成");
        orderList.add(orderDemo);
        orderDemo = new OrderStep();
        orderDemo.setOrderId(15103111039L);
        orderDemo.setDesc("完成");
        orderList.add(orderDemo);
        return orderList;
    }
}
```

- 消费顺序消息

```
public class ConsumerInOrder {
    public static void main(String[] args) throws Exception {
        DefaultMQPushConsumer consumer = new
            DefaultMQPushConsumer("consumer_group_name");
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        /**
* 设置Consumer第一次启动是从队列头部开始消费还是队列尾部开始消费
* 如果非第一次启动，那么按照上次消费的位置继续消费
*/
        consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET);
        consumer.subscribe("TopicTest", "TagA || TagC || TagD");
        // 设置消费者所属中心
        // consumer.setAz("az1");
        consumer.registerMessageListener(new MessageListenerOrderly() {
            Random random = new Random();
            @Override
            public ConsumeOrderlyStatus consumeMessage(List<MessageExt> msgs,
                                                       ConsumeOrderlyContext
context) {
                context.setAutoCommit(true);
                for (MessageExt msg : msgs) {
                    // 可以看到每个queue有唯一的consume线程来消费,
                    订单对每个queue(分区)有序
                        System.out.println("consumeThread=" +
                                           Thread.currentThread().getName() +
                                           "queueId=" + msg.getQueueId() + ",
                                           content:" + new
String(msg.getBody()));
                                           }
                                           try {
                                               //模拟业务逻辑处理中...

TimeUnit.SECONDS.sleep(random.nextInt(10));
                                           } catch (Exception e) {
                                               e.printStackTrace();
                                           }
                                           return ConsumeOrderlyStatus.SUCCESS;
                                           }
                                           });
                    consumer.start();
                    System.out.println("Consumer Started.");
                }
            }
```

- **3.6.3.2 延迟消息**
  - 发送延迟消息

```
public class ScheduledMessageProducer {
    public static void main(String[] args) throws Exception {
        // 实例化一个生产者来产生延时消息
        DefaultMQProducer producer = new
```

```
            DefaultMQProducer("ExampleProducerGroup");
        // 设置Coordinator的地址
        producer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置生产者所属中心
        // producer.setAz("az1");
        // 启动生产者
        producer.start();
        int totalMessagesToSend = 100;
        for (int i = 0; i < totalMessagesToSend; i++) {
            Message message = new Message("TestTopic", ("Hello scheduled message
                                                        " + i).getBytes());
                                                        // 延迟等级："1s 5s 10s
30s 1m 2m 3m 4m 5m 6m 7m 8m 9m 10m 20m 30m 1h
                                                        2h"
                                                        // 设置延时等级3,这个消息将在
10s之后发送

 message.setDelayTimeLevel(3);
                                                        // 设置延迟10000毫秒后投递消
息
                                                        //
message.setDelayTimeMs(10000);
                                                        // 设置延迟10秒后投递消息
                                                        //
message.setDelayTimeSec(10);
                                                        // 设置任意时间毫秒投递消息
                                                        //
message.setDeliverTimeMs(System.currentTimeMillis() + 10000);
                                                        // 发送消息
                                                        producer.send(message);
                                                        }
                                                        // 关闭生产者
                                                        producer.shutdown();
                                                        }
                                                        }
```

- 消费延时消息

```
public class ScheduledMessageConsumer {
    public static void main(String[] args) throws Exception {
        // 实例化消费者
        DefaultMQPushConsumer consumer = new
            DefaultMQPushConsumer("ExampleConsumer");
        // 设置Coordinator的地址
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置消费者所属中心
        // consumer.setAz("az1");
        // 订阅Topics
        consumer.subscribe("TestTopic", "*");
        // 注册消息监听者
        consumer.registerMessageListener(new MessageListenerConcurrently() {
```

```
            @Override
            public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>
                                                            messages,
ConsumeConcurrentlyContext context) {
                for (MessageExt message : messages) {
                    // 打印一条消息ID和延迟的时间
                    System.out.println("Receive message[msgId=" +
                                       message.getMsgId() + "] " +
(System.currentTimeMillis() -

 message.getBornTimestamp()) + "ms later");
                }
                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
            }
        });
        // 启动消费者
        consumer.start();
    }
}
```

- **3.6.3.3 批量消息**
  - 发送批量消息

```
public class SimpleBatchProducer {
    public static final String PRODUCER_GROUP = "BatchProducerGroupName";
    public static final String DEFAULT_COORDINATOR_ADDR = "127.0.0.1:8156";
    public static final String TOPIC = "BatchTest";
    public static final String TAG = "Tag";
    public static void main(String[] args) throws Exception {
        DefaultMQProducer producer = new DefaultMQProducer(PRODUCER_GROUP);
        producer.setCoordinatorAddr(DEFAULT_COORDINATOR_ADDR);
        // 设置生产者所属中心
        // producer.setAz("az1");
        producer.start();
        //将一批消息放入list中，使用一个请求发送
        List<Message> messages = new ArrayList<>();
        messages.add(new Message(TOPIC, TAG, "OrderID001", "Hello world
                                 0".getBytes()));
                                 messages.add(new Message(TOPIC, TAG,
"OrderID002", "Hello world
                                                          1".getBytes()));
                                                          messages.add(new
Message(TOPIC, TAG, "OrderID003", "Hello world

  2".getBytes()));

  SendResult sendResult = producer.send(messages);

  System.out.printf("%s", sendResult);

  }
```

```
  }
```

- 消息列表分割

```
public class SplitBatchProducer {
    public static final String PRODUCER_GROUP = "BatchProducerGroupName";
    public static final String DEFAULT_COORDINATOR_ADDR = "127.0.0.1:8156";
    public static final int MESSAGE_COUNT = 100 * 1000;
    public static final String TOPIC = "BatchTest";
    public static final String TAG = "Tag";
    public static void main(String[] args) throws Exception {
        DefaultMQProducer producer = new DefaultMQProducer(PRODUCER_GROUP);
        producer.setCoordinatorAddr(DEFAULT_COORDINATOR_ADDR);
        // 设置生产者所属中心
        // producer.setAz("az1");
        producer.start();
        List<Message> messages = new ArrayList<>(MESSAGE_COUNT);
        for (int i = 0; i < MESSAGE_COUNT; i++) {
            messages.add(new Message(TOPIC, TAG, "OrderID" + i, ("Hello world "
                                                                 +
i).getBytes()));
        }
        //将一大批消息分隔成一小批消息
        ListSplitter splitter = new ListSplitter(messages);
        while (splitter.hasNext()) {
            List<Message> listItem = splitter.next();
            SendResult sendResult = producer.send(listItem);
            System.out.printf("%s", sendResult);
        }
    }
}
class ListSplitter implements Iterator<List<Message>> {
    private static final int SIZE_LIMIT = 1000 * 1000;
    private final List<Message> messages;
    private int currIndex;
    public ListSplitter(List<Message> messages) {
        this.messages = messages;
    }
    @Override
    public boolean hasNext() {
        return currIndex < messages.size();
    }
    @Override
    public List<Message> next() {
        int nextIndex = currIndex;
        int totalSize = 0;
        for (; nextIndex < messages.size(); nextIndex++) {
            Message message = messages.get(nextIndex);
            int tmpSize = message.getTopic().length() +
                message.getBody().length;
```

```
            Map<String, String> properties = message.getProperties();
            for (Map.Entry<String, String> entry : properties.entrySet()) {
                tmpSize += entry.getKey().length() + entry.getValue().length();
            }
            tmpSize = tmpSize + 20;
            if (tmpSize > SIZE_LIMIT) {
                if (nextIndex - currIndex == 0) {
                    nextIndex++;
                }
                break;
            }
            if (tmpSize + totalSize > SIZE_LIMIT) {
                break;
            } else {
                totalSize += tmpSize;
            }
        }
        List<Message> subList = messages.subList(currIndex, nextIndex);
        currIndex = nextIndex;
        return subList;
    }
    @Override
    public void remove() {
        throw new UnsupportedOperationException("Not allowed to remove");
    }
}
```

- **3.6.3.4 消息过滤**
  - **3.6.3.4.1 TAG过滤**

    在大多数情况下，TAG是一个简单而有用的设计，其可以来选择您想要的消息。例如：

    ```
    DefaultMQPushConsumer consumer = new DefaultMQPushConsumer("CID_EXAMPLE");
    consumer.subscribe("TOPIC", "TAGA || TAGB || TAGC");
    ```

    消费者将接收包含TAGA或TAGB或TAGC的消息。

    - 发送消息-TAG过滤

      ```
      public class TagFilterProducer {
          public static void main(String[] args) throws Exception {
              DefaultMQProducer producer = new
                  DefaultMQProducer("producer_group_name");
              // 设置Coordinator的地址
              producer.setCoordinatorAddr("127.0.0.1:8156");
              // 设置生产者所属中心
              // producer.setAz("az1");
              producer.start();
              String[] tags = new String[] {"TagA", "TagB", "TagC"};
              for (int i = 0; i < 60; i++) {
                  Message msg = new Message("TagFilterTest",
      ```

```
                                        tags[i % tags.length],
                                        "Hello
world".getBytes(RemotingHelper.DEFAULT_CHARSET));
            SendResult sendResult = producer.send(msg);
            System.out.printf("%s%n", sendResult);
        }
        producer.shutdown();
    }
}
```

- 消费消息-TAG过滤

```
public class TagFilterConsumer {
    public static void main(String[] args) throws InterruptedException,
    MQClientException, IOException {
        DefaultMQPushConsumer consumer = new
            DefaultMQPushConsumer("consumer_group_name");
        // 设置Coordinator的地址
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        consumer.subscribe("TagFilterTest", "TagA || TagC");
        // 设置消费者所属中心
        // consumer.setAz("az1");
        consumer.registerMessageListener(new MessageListenerConcurrently() {
            @Override
            public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>
                                                            msgs,

 ConsumeConcurrentlyContext context) {
                System.out.printf("%s Receive New Messages: %s %n",
                                  Thread.currentThread().getName(), msgs);
                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
            }
        });
        consumer.start();
        System.out.printf("Consumer Started.%n");
    }
}
```

- **3.6.3.4.2 SQL92过滤过滤**

TAG过滤的限制是一个消息只能有一个标签，对于复杂场景可以使用SQL表达式筛选消息。

- 发送消息-SQL92过滤

发送消息时，通过putUserProperty来设置消息的属性

```
public class SqlFilterProducer {
    public static void main(String[] args) throws Exception {
        DefaultMQProducer producer = new
            DefaultMQProducer("producer_group_name");
        // 设置Coordinator的地址
        producer.setCoordinatorAddr("127.0.0.1:8156");
```

```
        // 设置生产者所属中心
        // producer.setAz("az1");
        producer.start();
        String[] tags = new String[] {"TagA", "TagB", "TagC"};
        for (int i = 0; i < 10; i++) {
            Message msg = new Message("SqlFilterTest",
                                      tags[i % tags.length],
                                      ("Hello BES CloudMQ " +
i).getBytes(RemotingHelper.DEFAULT_CHARSET)
                                      );
            msg.putUserProperty("a", String.valueOf(i));
            SendResult sendResult = producer.send(msg);
            System.out.printf("%s%n", sendResult);
        }
        producer.shutdown();
    }
}
```

- 消费消息-SQL92过滤

```
public class SqlFilterConsumer {
    public static void main(String[] args) throws Exception {
        DefaultMQPushConsumer consumer = new
            DefaultMQPushConsumer("consumer_group_name");
        // 设置Coordinator的地址
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置消费者所属中心
        // consumer.setAz("az1");
        // 消费者通过sql92的表达方式进行订阅
        consumer.subscribe("SqlFilterTest",
                           MessageSelector.bySql("(TAGS is not null and TAGS
in ('TagA',
                                                 'TagB'))" +
            "and (a is not null and a between 0 and 3)"));
        consumer.registerMessageListener(new MessageListenerConcurrently() {
            @Override
            public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>
                                                            msgs,
ConsumeConcurrentlyContext context) {
                System.out.printf("%s Receive New Messages: %s %n",
                                  Thread.currentThread().getName(), msgs);
                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
            }
        });
        consumer.start();
        System.out.printf("Consumer Started.%n");
    }
}
```

- **3.6.3.5 事务消息**

事务消息共有三种状态，提交状态、回滚状态、中间状态：

1. TransactionStatus.CommitTransaction: 提交事务，它允许消费者消费此消息
2. TransactionStatus.RollbackTransaction: 回滚事务，它代表该消息将被删除，不允许被消费。
3. TransactionStatus.Unknown: 中间状态，它代表需要检查消息队列来确定状态。

- 发送事务消息

  （1）创建事务消息生产者

  使用TransactionMQProducer类创建生产者，并指定唯一的ProducerGroup，就可以设置自定义线程池来处理这些检查请求。执行本地事务后、需要根据执行结果对消息队列进行回复。

```
public class TransactionProducer {
    public static void main(String[] args) throws MQClientException,
    InterruptedException {
        TransactionListener transactionListener = new TransactionListenerImpl();
        TransactionMQProducer producer = new
            TransactionMQProducer("producer_group_name",
Arrays.asList("TopicTest1234"));
        ExecutorService executorService = new ThreadPoolExecutor(2, 5, 100,

TimeUnit.SECONDS, new ArrayBlockingQueue<Runnable>(2000), new
                                                                ThreadFactory()
{
                                                                    @Override
                                                                    public

Thread newThread(Runnable r) {
                                                                        Thread

thread = new Thread(r);

thread.setName("client-transaction-msg-check-thread");
                                                                        return

thread;
                                                                    }
                                                                });
        // 设置Coordinator的地址
        producer.setCoordinatorAddr("127.0.0.1:8156");
        producer.setExecutorService(executorService);
        producer.setTransactionListener(transactionListener);
        // 设置生产者所属中心
        // producer.setAz("az1");
        producer.start();
        String[] tags = new String[] {"TagA", "TagB", "TagC", "TagD", "TagE"};
        for (int i = 0; i < 10; i++) {
            try {
                Message msg =
                    new Message("TopicTest1234", tags[i % tags.length], "KEY" +
                                i,
                                ("Hello BES CloudMQ " +
                                 i).getBytes(RemotingHelper.DEFAULT_CHARSET));
                SendResult sendResult = producer.sendMessageInTransaction(msg,
```

```
                                                                    null);
                System.out.printf("%s%n", sendResult);
                Thread.sleep(10);
            } catch (MQClientException | UnsupportedEncodingException e) {
                e.printStackTrace();
            }
        }
        for (int i = 0; i < 100000; i++) {
            Thread.sleep(1000);
        }
        producer.shutdown();
    }
}
```

（2）实现事务的监听接口

当发送半消息成功时，我们使用executeLocalTransaction 方法来执行本地事务。它返回前一节中提到的三个事务状态之一。checkLocalTransaction 方法用于检查本地事务状态，并回应消息队列的检查请求。它也是返回前一节中提到的三个事务状态之一。

```
public class TransactionListenerImpl implements TransactionListener {
    private AtomicInteger transactionIndex = new AtomicInteger(0);
    private ConcurrentHashMap<String, Integer> localTrans = new
        ConcurrentHashMap<>();
    //执行本地事务
    @Override
    public LocalTransactionState executeLocalTransaction(Message msg, Object arg)
    {
        //执行本地事务
        int value = transactionIndex.getAndIncrement();
        int status = value % 3;
        localTrans.put(msg.getTransactionId(), status);
        //返回本地事务执行结果
        return LocalTransactionState.UNKNOW;
    }
    //检查本地事务执行结果
    @Override
    public LocalTransactionState checkLocalTransaction(MessageExt msg) {
        Integer status = localTrans.get(msg.getTransactionId());
        if (null != status) {
            switch (status) {
                case 0:
                    return LocalTransactionState.UNKNOW;
                case 1:
                    return LocalTransactionState.COMMIT_MESSAGE;
                case 2:
                    return LocalTransactionState.ROLLBACK_MESSAGE;
            }
        }
        //返回本地事务执行结果
        return LocalTransactionState.COMMIT_MESSAGE;
    }
```

```
}
```

- 事务消息限制

1. 事务消息不支持延时消息和批量消息。
2. 事务消息将在Broker 配置文件中的参数transactionTimeout 这样的特定时间长度之后被检查。当发送事务消息时，用户还可以通过设置用户属性CHECK_IMMUNITY_TIME_IN_SECONDS 来改变这个限制，该参数优先于transactionTimeout 参数。
3. 事务性消息可能不止一次被检查或消费。
4. 事务消息的生产者组名不能与其他类型消息的生产者组名共享。与其他类型的消息不同，事务消息允许反向查询、MQ服务器能通过它们的生产者组名查询到在线事物生产者的监听接口。

- **3.6.3.6 RPC调用方式发送消息**

RPC调用方式发送消息，即生产者发送一条消息，消费者消费消息后会向生产者端发送一条reply消息，实现rpc调用效果。

- 生产者

```
public class AsyncRequestProducer {
    private static final InternalLogger log = ClientLogger.getLog();
    public static void main(String[] args) throws MQClientException,
    InterruptedException {
        String producerGroup = "producer_group_name";
        String topic = "RequestTopic";
        long ttl = 3000;
        DefaultMQProducer producer = new DefaultMQProducer(producerGroup);
        producer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置生产者所属中心
        // producer.setAz("az1");
        producer.start();
        try {
            Message msg = new Message(topic,
                                      "Hello
world".getBytes(RemotingHelper.DEFAULT_CHARSET));
            long begin = System.currentTimeMillis();
            producer.request(msg, new RequestCallback() {
                @Override
                public void onSuccess(Message message) {
                    //接收消费者的reply消息
                    long cost = System.currentTimeMillis() - begin;
                    System.out.printf("request to <%s> cost: %d replyMessage:
                                      %s %n", topic, cost, message);
                                      }
                                      @Override
                                      public void onException(Throwable e) {
                                          System.err.printf("request to <%s>
fail.", topic);
                                      }
                                      }, ttl);
                } catch (Exception e) {
```

```
                    log.warn("", e);
                }
            }
                                }
```

- 消费者

```
public class ResponseConsumer {
    public static void main(String[] args) throws InterruptedException,
    MQClientException {
        String producerGroup = "producer_group_name";
        String consumerGroup = "consumer_group_name";
        String topic = "RequestTopic";
        // 创建生产者用于发送reply消息
        DefaultMQProducer replyProducer = new DefaultMQProducer(producerGroup);
        replyProducer.setCoordinatorAddr("127.0.0.1:8156");
        replyProducer.start();
        // 创建消费者
        DefaultMQPushConsumer consumer = new
            DefaultMQPushConsumer(consumerGroup);
        consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_LAST_OFFSET);
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置异常时的延迟时间
        consumer.setPullTimeDelayMillsWhenException(0L);
        // 设置消费者所属中心
        // consumer.setAz("az1");
        consumer.registerMessageListener(new MessageListenerConcurrently() {
            @Override
            public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>
                                                            msgs,
ConsumeConcurrentlyContext context) {
                //接收生产者端发送的消息
                System.out.printf("%s Receive New Messages: %s %n",
                                  Thread.currentThread().getName(), msgs);
                for (MessageExt msg : msgs) {
                    try {
                        System.out.printf("handle message: %s %n",
                                          msg.toString());
                        // 向生产者端发送reply消息
                        String replyTo = MessageUtil.getReplyToClient(msg);
                        byte[] replyContent = "reply message
                            contents.".getBytes();
                            // 创建一个reply消息
                            Message replyMessage =
                            MessageUtil.createReplyMessage(msg, replyContent);
                        SendResult replyResult =
                            replyProducer.send(replyMessage, 3000);
                        System.out.printf("reply to %s , %s %n", replyTo,
                                          replyResult.toString());
                    } catch (MQClientException | RemotingException |
                             MQBrokerException | InterruptedException e) {
```

```
                        e.printStackTrace();
                    }
                }
                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
            }
        });
        consumer.subscribe(topic, "*");
        consumer.start();
        System.out.printf("Consumer Started.%n");
    }
}
```

- **3.6.3.7 OpenMessaging**

  OpenMessaging旨在为金融、电子商务、物联网和大数据领域提供通用框架及工业级指导方案。在分布式异构环境中，设计原则是面向云、简单、灵活和独立于语言。符合这些规范将帮助企业方便的开发跨平台和操作系统的异构消息传递应用程序。BES CloudMQ中提供了openmessaging-api 0.3.0的部分实现，需额外引入openmessaging-api-0.3.1-alpha.jar包。

  - Producer

```
public class SimpleProducer {
    //127.0.0.1:8156为协调实例地址
    public static final String URL =
        "oms:cloudmq.amphi://127.0.0.1:8156/default:default";
    //主题
    public static final String QUEUE = "clu1_topic_1";
    public static void main(String[] args) {
        //设置环境变量OMS_CMQ_DIRECT_NAME_SRV=true
        final MessagingAccessPoint messagingAccessPoint =
            OMS.getMessagingAccessPoint(URL);
        final Producer producer = messagingAccessPoint.createProducer();
        messagingAccessPoint.startup();
        System.out.printf("MessagingAccessPoint startup OK%n");
        producer.startup();
        System.out.printf("Producer startup OK%n");
        {
            Message message = producer.createBytesMessage(QUEUE,

 "OMS_HELLO_BODY".getBytes());
            SendResult sendResult = producer.send(message);
            System.out.printf("Send async message OK, msgId: %s%n",
                              sendResult.messageId());
        }
        final CountDownLatch countDownLatch = new CountDownLatch(1);
        {
            final Future<SendResult> result =
                producer.sendAsync(producer.createBytesMessage(QUEUE,

"OMS_HELLO_BODY".getBytes()));
            result.addListener(future -> {
                if (future.getThrowable() != null) {
```

```
                    System.out.printf("Send async message Failed, error: %s%n",
                                      future.getThrowable().getMessage());
                } else {
                    System.out.printf("Send async message OK, msgId: %s%n",
                                      future.get().messageId());
                }
                countDownLatch.countDown();
            });
        }
        {
            producer.sendOneway(producer.createBytesMessage("OMS_HELLO_TOPIC",

 "OMS_HELLO_BODY".getBytes()));
            System.out.printf("Send oneway message OK%n");
        }
        try {
            countDownLatch.await();
            Thread.sleep(500);
        } catch (InterruptedException ignore) {
        }
        producer.shutdown();
    }
}
```

- PullConsumer

  OpenMessaging 创建PullConsumer ， 从指定的队列中拉取消息

```
public class SimplePullConsumer {
    //127.0.0.1:8156为协调实例地址
    public static final String URL =
        "oms:cloudmq.amphi://127.0.0.1:8156/default:default";
    public static final String QUEUE = "clu1_topic_1"; //主题名称
    public static void main(String[] args) {
        //设置环境变量OMS_CMQ_DIRECT_NAME_SRV=true
        final MessagingAccessPoint messagingAccessPoint =
            OMS.getMessagingAccessPoint(URL);
        messagingAccessPoint.startup();
        final Producer producer = messagingAccessPoint.createProducer();
        //指定订阅组clu1_group_2，创建消费者
        final PullConsumer consumer = messagingAccessPoint.createPullConsumer(
            OMS.newKeyValue().put(OMSBuiltinKeys.CONSUMER_ID, "clu1_group_2"));
        messagingAccessPoint.startup();
        System.out.printf("MessagingAccessPoint startup OK%n");
        final String queueName = QUEUE;
        producer.startup();
        Message msg = producer.createBytesMessage(queueName, "Hello Open
                                                   Messaging".getBytes());
                                                   SendResult sendResult =
producer.send(msg);
                                                   System.out.printf("Send Message
OK. MsgId: %s%n",
```

```
 sendResult.messageId());
                                                    producer.shutdown();

 consumer.attachQueue(queueName);
                                                    consumer.startup();
                                                    System.out.printf("Consumer
startup OK%n");
                                                    boolean stop = false;
                                                    while (!stop) {
                                                        //拉取消息
                                                        Message message =
consumer.receive();
                                                        if (message != null) {
                                                            String msgId =

 message.sysHeaders().getString(Message.BuiltinKeys.MESSAGE_ID);

 System.out.printf("Received one message: %s%n", msgId);
                                                                consumer.ack(msgId);
                                                                if (!stop) {
                                                                    stop =
msgId.equalsIgnoreCase(sendResult.messageId());
                                                                }
                                                        } else {

 System.out.printf("Return without any message%n");
                                                        }
                                                    }
                                                    consumer.shutdown();

 messagingAccessPoint.shutdown();
                                                    }
                                                    }
```

- PushConsumer

  OpenMessaging 创建PushConsumer ，添加到指定的主题，并通过MessageListener 消费消息。

```
public class SimplePushConsumer {
    //127.0.0.1:8156为协调实例地址
    public static final String URL =
        "oms:cloudmq.amphi://127.0.0.1:8156/default:default";
    //主题名称
    public static final String QUEUE = "clu1_topic_1";
    public static void main(String[] args) {
        // 设置环境变量OMS_CMQ_DIRECT_NAME_SRV=true
        final MessagingAccessPoint messagingAccessPoint = OMS
            .getMessagingAccessPoint(URL);
        // 指定订阅组clu1_group_2，创建消费者
        final PushConsumer consumer = messagingAccessPoint.
            createPushConsumer(OMS.newKeyValue().put(OMSBuiltinKeys.CONSUMER_ID
```

```
                                                    ,
                                                    "clu1_group_2"));
        messagingAccessPoint.startup();
        System.out.printf("MessagingAccessPoint startup OK%n");
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            consumer.shutdown();
            messagingAccessPoint.shutdown();
        }));
        //监听消费消息
        consumer.attachQueue(QUEUE, (message, context) -> {
            System.out.printf("Received one message: %s%n",

 message.sysHeaders().getString(Message.BuiltinKeys.MESSAGE_ID));
            context.ack();
        });
        consumer.startup();
        System.out.printf("Consumer startup OK%n");
    }
}
```

3.6.4 实践示例

以下代码只是示例，其中配置根据实际情况进行修改，建议将配置写到yaml文件或者启动脚本中。

- Java MQ 客户端
  - 生产者Demo
    - 三种发送方式
      - 同步发送：这种发送方式可靠性最好，适用于比较重要的消息，如重要的消息通知，短信通知。

```
public class SyncProducer {
    public static void main(String[] args) throws Exception {
        // 实例化消息生产者Producer
        DefaultMQProducer producer = new
DefaultMQProducer("producer_group_name");
        // 设置Coordinator的地址
        producer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置生产者所属中心
        // producer.setAz("az1");
        // 启动Producer实例
        producer.start();
        for (int i = 0; i < 100; i++) {
            // 创建消息，并指定Topic，Tag和消息体
            Message msg = new Message("TopicTest" /* Topic */,
                                      "TagA" /* Tag */,("Hello BES
CloudMQ " +

 i).getBytes(RemotingHelper.DEFAULT_CHARSET)
                                      );
            // 发送消息到一个Broker
```

```
            SendResult sendResult = producer.send(msg);
            // 通过sendResult返回消息是否成功送达
            System.out.printf("%s%n", sendResult);
        }
        // 如果不再发送消息，关闭Producer实例。
        producer.shutdown();
    }
}
```

- 异步发送：异步消息通常用在对响应时间敏感的业务场景，即发送端不能容忍长时间地等待Broker的响应。

```
public class AsyncProducer {
    public static void main(String[] args) throws Exception {
        // 实例化消息生产者Producer
        DefaultMQProducer producer = new
            DefaultMQProducer("producer_group_name");
        // 设置Coordinator的地址
        producer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置生产者所属中心
        // producer.setAz("az1");
        // 启动Producer实例
        producer.start();
        producer.setRetryTimesWhenSendAsyncFailed(0);
        int messageCount = 100;
        // 根据消息数量实例化倒计时计算器
        final CountDownLatch2 countDownLatch = new
            CountDownLatch2(messageCount);
        for (int i = 0; i < messageCount; i++) {
            final int index = i;
            // 创建消息，并指定Topic，Tag和消息体
            Message msg = new Message("TopicTest",
                                      "TagA",
                                      "OrderID188","Hello
world".getBytes(RemotingHelper.DEFAULT_CHARSET));
            // SendCallback接收异步返回结果的回调
            producer.send(msg, new SendCallback() {
                @Override
                public void onSuccess(SendResult sendResult) {
                    countDownLatch.countDown();
                    System.out.printf("%-10d OK %s %n", index,
                                      sendResult.getMsgId());
                }
                @Override
                public void onException(Throwable e) {
                    countDownLatch.countDown();
                    System.out.printf("%-10d Exception %s %n", index,
e);
                    e.printStackTrace();
                }
            });
```

```
        }
        // 等待5s
        countDownLatch.await(5, TimeUnit.SECONDS);
        // 如果不再发送消息，关闭Producer实例。
        producer.shutdown();
    }
}
```

- OneWay发送：这种方式主要用在不特别关心发送结果的场景，例如日志发送。

```
public class OnewayProducer {
    public static void main(String[] args) throws Exception{
        // 实例化消息生产者Producer
        DefaultMQProducer producer = new
            DefaultMQProducer("producer_group_name");
        // 设置Coordinator的地址
        producer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置生产者所属中心
        // producer.setAz("az1");
        // 启动Producer实例
        producer.start();
        for (int i = 0; i < 100; i++) {
            // 创建消息，并指定Topic，Tag和消息体
            Message msg = new Message("TopicTest" /* Topic */,"TagA" /*
Tag */,
                                      ("Hello BES CloudMQ " + i)

.getBytes(RemotingHelper.DEFAULT_CHARSET)
                                     );
            // 发送单向消息，没有任何返回结果
            producer.sendOneway(msg);
        }
        // 如果不再发送消息，关闭Producer实例。
        producer.shutdown();
    }
}
```

- 消费者Demo
  - 集群Push消费

```
public class PushConsumer {
    public static void main(String[] args) throws InterruptedException,
    MQClientException {
        // 实例化消费者
        DefaultMQPushConsumer consumer =
            new DefaultMQPushConsumer("consumer_group_name");
        // 设置Coordinator的地址
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        //设置集群消费，也可不设置，默认就是集群消费
        consumer.setMessageModel(MessageModel.CLUSTERING);
```

```
        // 设置消费者所属中心
        // consumer.setAz("az1");
        // 订阅一个或者多个Topic，以及Tag来过滤需要消费的消息
        consumer.subscribe("TopicTest", "*");
        // 注册回调实现类来处理从broker拉取回来的消息
        consumer.registerMessageListener(new MessageListenerConcurrently() {
            @Override
            public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>

 msgs,ConsumeConcurrentlyContext context) {
                System.out.printf("%s Receive New Messages: %s %n",
                                  Thread.currentThread().getName(), msgs);
                // 标记该消息已经被成功消费
                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
            }
        });
        // 启动消费者实例
        consumer.start();
        System.out.printf("Consumer Started.%n");
    }
}
```

- 广播Push消费

```
public class PushConsumer {
    public static void main(String[] args) throws InterruptedException,
    MQClientException {
        DefaultMQPushConsumer consumer = new
            DefaultMQPushConsumer("consumer_group_name");
        consumer.setCoordinatorAddr("127.0.0.1:8156");

 consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET);
        consumer.setMessageModel(MessageModel.BROADCASTING);
        consumer.subscribe("TopicTest", "*");
        consumer.registerMessageListener(new MessageListenerConcurrently() {
            @Override
            public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>
                                                            msgs,
ConsumeConcurrentlyContext context) {
                System.out.printf("%s Receive New Messages: %s %n",
                                  Thread.currentThread().getName(), msgs);
                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
            }
        });
        consumer.start();
        System.out.printf("Consumer Started.%n");
    }
}
```

- Pull消费

```
public class PullConsumer {
    public static void main(String[] args) throws MQClientException {
        // 创建PullConsumer
        DefaultMQPullConsumer consumer = new
            DefaultMQPullConsumer("consumer_group_name");
        // 设置Coordinator的地址
        consumer.setCoordinatorAddr("127.0.0.1:8156");
        // 注册需要拉取的Topic集合
        Set<String> topics = new HashSet<>();
        topics.add("TopicTest");
        consumer.setRegisterTopics(topics);
        // 启动消费者
        consumer.start();
        // 按Topic数量创建线程池，每个Topic分配一个拉取线程
        ExecutorService executors = Executors.newFixedThreadPool(
            topics.size(),
            new ThreadFactoryImpl("PullConsumerThread")
        );
        // 遍历已注册的Topic，为每个Topic启动一个独立拉取任务
        for (String topic : consumer.getRegisterTopics()) {
            executors.execute(new Runnable() {
                public void doSomething(List<MessageExt> msgs) {
                    // 处理业务逻辑
                }
                @Override
                public void run() {
                    while (true) {
                        try {
                            // 获取当前消费者在该Topic下分配到的消息队列
                            Set<MessageQueue> messageQueues =
                                consumer.fetchMessageQueuesInBalance(topic);
                            // 当前没有分配到队列时，等待后重试
                            if (messageQueues == null ||
                                messageQueues.isEmpty()) {
                                Thread.sleep(1000);
                                continue;
                            }
                            PullResult pullResult = null;
                            // 遍历当前分配到的每个消息队列，逐个进行拉取
                            for (MessageQueue messageQueue : messageQueues) {
                                try {
                                    // 获取当前队列的消费起始偏移量
                                    long offset =
                                        this.consumeFromOffset(messageQueue);
                                    // 从指定队列、指定偏移量开始拉取消息，单次最多
拉取
                                    32 条
                                        pullResult =
consumer.pull(messageQueue,
                                                                    "*",
offset, 32);
                                    // 根据拉取结果状态做不同处理
```

```
                                switch (pullResult.getPullStatus()) {
                                    case FOUND:
                                        // 拉取到消息
                                        List<MessageExt> msgs =
                                            pullResult.getMsgFoundList();
                                        if (msgs != null &&
                                            !msgs.isEmpty()) {
                                            // 执行业务处理
                                            this.doSomething(msgs);
                                            // 更新当前队列的消费进度到本地内
存，
                                                后续会持久化到Broker

 consumer.updateConsumeOffset(
                                                    messageQueue,

 pullResult.getNextBeginOffset()
                                            );
                                            // 统计拉取TPS
                                            this.incPullTPS(topic,
pullResu ]

 lt.getMsgFoundList().size());
                                        }
                                        break;
                                    case OFFSET_ILLEGAL:
                                        // 偏移量非法，
                                        使用Broker返回的新偏移量进行纠正
                                            consumer.updateConsumeOffset(
                                            messageQueue,

 pullResult.getNextBeginOffset()
                                        );
                                        break;
                                    case NO_NEW_MSG:
                                        // 没有新消息，短暂休眠后继续拉取，
                                        同时推进偏移量
                                            Thread.sleep(1);
                                        consumer.updateConsumeOffset(
                                            messageQueue,

 pullResult.getNextBeginOffset()
                                        );
                                        break;
                                    case NO_MATCHED_MSG:
                                       // 没有匹配到消息，更新偏移量继续下一轮
拉取
                                        consumer.updateConsumeOffset(
                                            messageQueue,

 pullResult.getNextBeginOffset()
                                        );
```

```
                                        break;
                                    default:
                                        // 其他状态不处理
                                }
                            } catch (RemotingException e) {
                                // 网络通信异常
                                e.printStackTrace();
                            } catch (MQBrokerException e) {
                                // Broker 返回异常
                                e.printStackTrace();
                            } catch (Exception e) {
                                // 其他异常
                                e.printStackTrace();
                            }
                        }
                    } catch (MQClientException e) {
                        // 负载均衡或客户端相关异常
                        e.printStackTrace();
                    } catch (InterruptedException e) {
                        // 线程中断异常
                        e.printStackTrace();
                    } catch (Exception e) {
                        // 其他未预期异常
                        e.printStackTrace();
                    }
                }
            }
            public long consumeFromOffset(MessageQueue messageQueue)
throws
                MQClientException {
                // 优先从本地内存读取消费进度
                long offset = consumer.getOffsetStore()
                    .readOffset(messageQueue,
                                ReadOffsetType.READ_FROM_MEMORY);
                if (offset < 0) {
                    // 本地内存没有时，再从Broker或本地持久化存储读取
                    offset = consumer.getOffsetStore()
                        .readOffset(messageQueue,
                                    ReadOffsetType.READ_FROM_STORE);
                }
                if (offset < 0) {
                    // 首次启动且没有历史消费位点时，从队列最大offset开始消费
                    offset = consumer.maxOffset(messageQueue);
                }
                if (offset < 0) {
                    // 兜底处理，避免出现非法偏移量
                    offset = 0;
                }
                return offset;
            }
            public void incPullTPS(String topic, int pullSize) {
                consumer.getDefaultMQPullConsumerImpl()
```

```
                        .getRebalanceImpl()
                        .getmQClientFactory()
                        .getConsumerStatsManager()
                        .incPullTPS(consumer.getConsumerGroup(), topic,
                                    pullSize);
                }
            });
        }
        // 关闭线程池
        executors.shutdown();
        // 关闭消费者
        consumer.shutdown();
    }
}
```

- 集群LitePull消费

```
public class LitePullConsumerSubscribe {
    public static volatile boolean running = true;
    public static void main(String[] args) throws Exception {
        DefaultLitePullConsumer litePullConsumer = new
            DefaultLitePullConsumer("clu1_group_1");

 litePullConsumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRS ]
                                              T_OFFSET);
        litePullConsumer.subscribe("clu1_topic_1", "*");
        litePullConsumer.setCoordinatorAddr("127.0.0.1:8156");
        // 设置消费者所属中心
        // litePullConsumer.setAz("az1");
        litePullConsumer.start();
        try {
            while (running) {
                //主动拉取消息
                List<MessageExt> messageExts = litePullConsumer.poll();
                if (!messageExts.isEmpty()) {
                    System.out.printf("%s%n", messageExts);
                }
            }
        } finally {
            litePullConsumer.shutdown();
        }
    }
}
```

- 集群LitePull指定队列消费：LitePull支持从指定的队列中拉取消息，支持设置队列的起始消费点位，支持手动提交offset。

```
public class LitePullConsumerAssign {
    public static volatile boolean running = true;
    public static void main(String[] args) throws Exception {
        DefaultLitePullConsumer litePullConsumer = new
```

```
            DefaultLitePullConsumer("clu1_group_1");
        litePullConsumer.setCoordinatorAddr("127.0.0.1:8156");
        //设置手动提交offset
        litePullConsumer.setAutoCommit(false);
        // 设置消费者所属中心
        // litePullConsumer.setAz("az1");
        litePullConsumer.start();
        Collection<MessageQueue> mqSet =
            litePullConsumer.fetchMessageQueues("clu1_topic_1");
        List<MessageQueue> list = new ArrayList<>(mqSet);
        List<MessageQueue> assignList = new ArrayList<>();
        for (int i = 0; i < list.size() / 2; i++) {
            assignList.add(list.get(i));
        }
        //指定队列列表
        litePullConsumer.assign(assignList);
        //指定队列的起始消费位置
        for (MessageQueue mq : assignList) {
            litePullConsumer.seek(mq, 0);
        }
        try {
            while (running) {
                List<MessageExt> messageExts = litePullConsumer.poll();
                System.out.printf("%s %n", messageExts);
                //手动提交offset
                litePullConsumer.commitSync();
            }
        } finally {
            litePullConsumer.shutdown();
        }
    }
}
```

3.7 可观测

3.7.1 功能说明

- 应用可观测

  与听云对接完成后，可通过本页面直接跳转听云APM，访问自身应用相关可观测数据。

- 集中日志

  日志采集接入成功后，可通过本菜单直接跳转访问采集到的应用日志。

- 监控指标(Metrics)

  基于Prometheus体系实现了对注册中心、配置中心、网关、缓存、消息队列的基础监控数据采集，此部分数据在相关产品部署时已自动完成采集配置，无需额外操作。可通过页面菜单 **运维监控-监控看板** 来查看相应的监控数据。

- 日志采集(Logs)

基于ELK实现了对应用日志的采集，针对K8s环境和虚机环境，需要部署不同的日志采集插件并进行相应的配置，具体请参见集中日志接入。

- 链路追踪(Traces)

  基于听云APM实现了应用的链路追踪能力，在应用部署时，需完成与听云的对接后，才能实现链路追踪能力，具体请参见应用可观测接入。

- 告警配置（建设中）

  基于Prometheus采集到的监控数据，按需自定义告警指标，实现告警通知和记录。

3.7.2 接入配置

- 应用可观测

  1. 联系 **听云APM** 负责人，安装、配置并启用可观测数据采集插件
  2. 访问磐石平台菜单 **应用可观测** ，系统将自动跳转到 **听云APM** 平台上并创建与磐石平台当前用户同名的用户 ，此时该用户尚无任何数据访问权限
  3. 联系 **听云APM** 负责人，为该用户赋权，设置为仅访问自身应用可观测数据
  4. 重新访问磐石平台菜单 **应用可观测**，系统将自动跳转到 **听云APM** 平台上并可以访问授权的数据

- 集中日志

  应用系统需已经注册了磐石平台用户，且获得了***tenantcode***、***workspacecode***和***projectcode***。
  磐石平台集中日志会将这三个的值转换为小写后，拼接组成唯一的日志索引，其格式为： ***tenantcode-workspacecode-projectcode***（后文简记作 ${index} ）。

  华为云集群部署日志抓取配置

  准备flunet-bit.4.0.3.tar镜像包和fluent-bit-0.54.0.tgz安装包

  控制台界面部署

```
1. 进入集群模版管理-导入模版 fluent-bit-0.54.0.tgz 安装包进行安装模板 fluentbit
2. 模版导入完成后安装模板
3. 进入工作负载-守护进程集-命名空间，找到该进程修改 ***spec.template.spac.contaierns.image***
参数为 flunet-bit.4.0.3.tar镜像包镜像包的地址
4. 进入配置项与密钥-命名空间，找到该配置项并编辑yaml
```

```
[SERVICE]
    Daemon Off
    Flush 1
    Log_Level info
    Parsers_File /fluent-bit/etc/parsers.conf
    Parsers_File /fluent-bit/etc/conf/custom_parsers.conf
    HTTP_Server On
    HTTP_Listen 0.0.0.0
    HTTP_Port 2020                          -----替换为实际端口
    Health_Check On
```

```
[INPUT]
    Name tail
    Path /var/log/containers/*.log          -----替换为具体的日志目录
    multiline.parser docker, cri
    Tag kube.*
    Mem_Buf_Limit 5MB
    Skip_Long_Lines On

[INPUT]
    Name systemd
    Tag host.*
    Systemd_Filter _SYSTEMD_UNIT=kubelet.service
    Read_From_Tail On

[FILTER]
    Name kubernetes
    Match kube.*
    Merge_Log On
    Keep_Log Off
    K8S-Logging.Parser On
    K8S-Logging.Exclude On

[OUTPUT]
    Name kafka
    Match kfffff.*                                      -----根据需求修改过滤条件
    Brokers x.x.x.x:port,x.x.x.x:port,x.x.x.x:port -----修改为实际IP和端口
    Topics ${index}                                     -----修改为获得的index值（小写）
    Retry_Limit 1
```

```
5. 重启进程
   <br>
```

集群服务器内部部署

1. 创建命名空间
   kubectl create ns logplatform -----命名空间可为任意符合规范的名称
2. 上传 flunet-bit.4.0.3.tar镜像包，将标签打为 ***cr.fluentbit.io/fluent/fluent-bit:4.1.0***
3. 部署安装
   helm install fluent fluent-bit-0.54.0.tgz -n logplatform
4. 修改配置文件并推送
   kubectl edit configmap fluentbit-fluent-bit -n logplatform

```
[SERVICE]
    Daemon Off
    Flush 1
    Log_Level info
    Parsers_File /fluent-bit/etc/parsers.conf
    Parsers_File /fluent-bit/etc/conf/custom_parsers.conf
    HTTP_Server On
```

```
    HTTP_Listen 0.0.0.0
    HTTP_Port 2020                              -----替换为实际端口
    Health_Check On

[INPUT]
    Name tail
    Path /var/log/containers/*.log          -----替换为具体的日志目录
    multiline.parser docker, cri
    Tag kube.*
    Mem_Buf_Limit 5MB
    Skip_Long_Lines On

[INPUT]
    Name systemd
    Tag host.*
    Systemd_Filter _SYSTEMD_UNIT=kubelet.service
    Read_From_Tail On

[FILTER]
    Name kubernetes
    Match kube.*
    Merge_Log On
    Keep_Log Off
    K8S-Logging.Parser On
    K8S-Logging.Exclude On

[OUTPUT]
    Name kafka
    Match kfffff.*                                          -----根据需求修改过滤条件
    Brokers x.x.x.x:port,x.x.x.x:port,x.x.x.x:port  -----修改为实际IP和端口
    Topics ${index}                                         -----修改为获得的index值（小写）
    Retry_Limit 1
```

虚拟机部署日志抓取配置

1. 准备包 filebeat-8.19.6-linux-arm64.tar
2. 修改配置文件

```
paths:
- /home/develop/*.log     -----替换为具体的日志目录

 output.elasticsearch:
 hosts: ["https://x.x.x.x:port","https://x.x.x.x:port","https://x.x.x.x:port"] -----替换为实际IP和端口
 ssl:
   verification_mode: none
 preset: balanced
 api_key: "id:${api_key}"            -----替换为实际elasticsearch的api_key
 index: "${index}-%{+YYYY.MM.dd}"    -----修改为获得的index值（小写）
 setup.ilm.enabled: false
 setup.template:
```

```
name: "${index}"                    ------修改为获得的index值（小写）
pattern: "${index}*"                ------修改为获得的index值（小写）
enable: true
overwrite: false
```

3. 启动filebeat
   nohup filebeat -e -c ./filebeat.yml &

---

4. 运维与支持

4.1 故障排查

4.1.1 服务注册失败排查

可能原因	排查步骤	解决方案
网络不通	检查服务器能否 telnet 注册中心端口	开通网络策略
命名空间/分组配置错误	检查 application.yml 中的 namespace 和 group	修正配置
服务名重复	检查注册中心中是否已有同名服务	更换服务名
SDK 版本问题	检查引入的 nacos-client 版本	升级到推荐版本

排查命令：

```
# 检查注册中心连通性
telnet $NAMING_IP 18662

# 查看本地 hosts 配置
cat /etc/hosts
```

4.1.2 配置推送失败排查

可能原因	排查步骤	解决方案
配置未发布	检查配置状态是否为"已发布"	在控制台发布配置
命名空间/分组不匹配	检查客户端配置的 namespace 和 group	修正配置
客户端未启用动态刷新	检查 spring.cloud.nacos.config.refresh-enabled	启用动态刷新
网络延迟	长轮询机制最大延迟约 3 秒	等待后重试

4.1.3 调用超时排查

可能原因	排查步骤	解决方案
被调用服务负载高	检查服务监控指标	联系被调用服务负责人
网络延迟/抖动	检查网络状态	联系网络团队排查
线程池满	检查应用线程池使用率	调整线程池大小或限流
GC 频繁	检查 JVM GC 指标	优化 JVM 参数

排查命令：

```
# 查看当前服务的线程堆栈
jstack <pid> > threaddump.txt

# 查看 JVM 内存使用
jmap -heap <pid>

# 查看 GC 情况
jstat -gc <pid> 1000 10
```

4.1.4 消息积压排查

可能原因	排查步骤	解决方案
消费者实例下线	检查消费者服务状态	重启消费者
消费逻辑耗时过长	检查单条消息处理时间	优化消费逻辑
消费者线程池满	检查消费者并发配置	调整消费线程数
消费抛出异常未重试	检查异常日志	修复消费逻辑

排查命令：

```
# 查看 Topic 消息堆积情况（通过 RocketMQ 控制台）
# 进入「消息队列」→「消息轨迹」，输入 MessageId 查询

# 查看消费者消费位点
cd $ROCKETMQ_HOME/bin
./mqadmin consumerProgress -n namesrv1:9876 -g consumerGroupName
```

---

5. FAQ

5.1 接入类

Q: 如何申请平台账号?

A: 在磐石平台控制台提交申请，具体见 2.1 业务系统平台使用申请。

5.2 配置类

Q: 配置中心配置迁移导入失败?

A: 常见原因及解决方式：

1. **DataId 格式错误**：确保文件名符合 `${service-name}-${env}.${file-type}` 格式
2. **Namespace 不存在**：先在控制台创建对应的 Namespace
3. **Group 不存在**：先在控制台创建对应的 Group
4. **文件编码问题**：确保文件编码为 UTF-8

Q: 配置变更如何生效?

A: 配置发布后，客户端通过长轮询机制感知变更（最长约 3 秒），自动拉取最新配置。如需强制刷新，可重启应用。

5.3 网关类

Q: 网关超时如何调整?

A:

1. 登录控制台，进入 **「API 网关管理」→「路由管理」**
2. 编辑目标路由规则
3. 在「超时配置」中设置超时时间（默认 5000ms）
4. 保存并发布

> 注意：超时时间不宜设置过长，建议根据业务实际耗时设置，避免大量请求堆积。

5.4 缓存/消息类

- Q: Redis 连接池满如何处理?

  根据项目业务需求，合理配置Redis客户端和服务端的配置，以保证连接池高效利用，避免连接池经常沾满或不够用。

  1. 客户端连接池配置。连接池大小要适中，避免过多或过少。配置合理的超时时间，设置适当的空闲连接管理。

     以lettuce为例，举例说明，仅供参考：

     ```
     spring:
       data:
         redis:
           # 具体的Redis配置...
           lettuce:
     ```

```
        pool:
          # 最大活跃连接数 - 根据应用并发量设置
          max-active: 20
          # 最大空闲连接数 - 一般为max-active的1/4到1/2
          max-idle: 10
          # 最小空闲连接数 - 保证基本连接数
          min-idle: 5
          # 连接超时时间
          max-wait: 2000ms
```

- max-active（最大活跃连接数）：
  推荐值：10-50
  如果应用并发量较高，可设置为30-50
  避免设置过大导致服务器压力
- max-idle（最大空闲连接数）：
  推荐值：5-15
  一般设置为max-active的1/4到1/2
- min-idle（最小空闲连接数）：
  推荐值：2-5
  保证在高并发时不会因连接不足影响响应
- max-wait（最大等待时间）：
  推荐值：2000ms
  避免请求长时间等待

2. 服务端配置

   评估项目使用缓存的业务场景，合理选择TCP keepalive间隔时间和客户端空闲超时时间。在申请集群时说明配置需求。

- Q: 死信消息产生后如何处理?

  死信消息（Dead Letter Message，简称DLQ）是一种特殊的消息处理机制，主要用于处理在正常消费过程中因为某些原因无法成功消费的消息。这些原因可能包括：

  - 消费失败：消费者在处理消息时发生异常，导致无法完成消费。
  - 处理超时：消息在消费者端处理超过最长时限而未成功消费。
  - 消费者未能确认消费：在某些情况下，消费者未能发送确认消息，导致消息未被认为已成功消费。

  被转移到死信队列中的消息（DLQ）可供后续进行人工干预或特殊处理。开发者或运维人员可以定期检查死信队列，以对未成功消费的消息进行分析、处理和重新消费。

  访问消息中间件集群所在管理端，查看死信消息步骤如下：

  1. 左侧导航区，点击死信消息，右侧显示死信消息页面。
  2. 下拉框中选择集群名称/订阅组名称，默认按消息ID查询，当按消息ID查询时，输入消息ID的内容，点击查询按钮，即可查找到对应的消息。
  3. 切换按订阅组查询，支持选择消息查询条数和时间框，点击查询按钮，即可查找到对应的消息。



###

6. 附录

暂无

本文档最后更新：2026年06月25日